Eidos: A Harness for Agents That Train Models

Why the scientific method belongs in the scaffold,
and what four development pilots taught us

Mrigesh Thakur

13 November 2026 · Exploratory development evidence · github.com/Legend101Zz/Eidos

Abstract. Language models now work inside the training loop: agents rewrite optimizers while a run is in progress, and models are reported to write network weights directly. But an agent is a model *plus its harness*, and the harnesses we have were built for software, where a test either passes or fails. Training is different. Feedback is noisy, every look at validation data spends some of its credibility, and the agent doing the search is tempted to grade it. We present **Eidos**, a harness that moves the discipline of empirical science into the scaffold: success rules are committed before results exist, search is kept apart from confirmation, every claim carries its scope, budgets are reserved rather than hoped for, and only an assessor with a separate identity can touch held-out data.

We test the idea directly. Agents share one model (MiniMax-M3.1-Flash) and one budget, and differ only in their harness; an independent grader on two fresh virtual machines decides every outcome. Over three pilots with four agents (132 runs) and four rounds of redesign, the Eidos investigator **matched the strongest coding agent, 12 of 12 verified improvements each, while using 5.5× fewer tokens**. In pilots 1 to 3 it **never claimed an improvement that the grader rejected**; all 14 of its conclusions in pilots 2 and 3 were confirmed. Its one setback, 2 of 12 against 11 of 12 on four newly built tasks, came from a single timing rule in its own runtime. Because every decision is on the record, the rule was easy to find, and the repaired build completed 7 of 7 runs where the old one completed 2. A fourth pilot, on four sealed tasks and two arms, then compared the repaired build with the coding agent that has the same toolkit. It was **inconclusive**: 20 of 24 verified improvements against 19 of 24, a difference of +0.042 with an exploratory 95% interval of [−0.208, +0.333]. Where both succeeded the gains were the same; the investigator used about 3.3× fewer processed tokens (usage uncertain on 17 of its 24 runs) but more wall time, and the sign of the difference depends on a registered run-status rule. Eidos's components help other agents too: given them as a toolkit, an ordinary coding agent was the strongest or joint-strongest arm in each of pilots 1 to 3. One expectation failed. Handing the full Eidos lab to a weak coding agent over MCP made it slower and costlier; in a small probe, Claude Opus 5.5 in the same role verified both of its tasks in about two minutes each, though it leaned on Eidos only lightly. The evidence is exploratory, and every defect we found along the way is on the public record.

1 Introduction

Something changed in 2026 in how people talk about training neural networks. For a decade, "automating training" meant searching a space someone had specified by hand: learning rates, schedules, architectures. Then language models moved into the loop itself. In one recent system, a pretrained language-model agent watches a run's training dynamics and rewrites the optimizer's executable code while the run is still going (Bai et al., 2026). Control planes now let agents adjust the hyperparameters, data and checkpoints of live training jobs through validated, logged actions (Zhang et al., 2025, 2026b; Rao and Advani, 2026). Language models are even reported to read and write the weights of small networks directly, with no gradient descent at all. That last claim is exciting, and fiercely contested (Takehi et al., 2026; Honnibal, 2026;

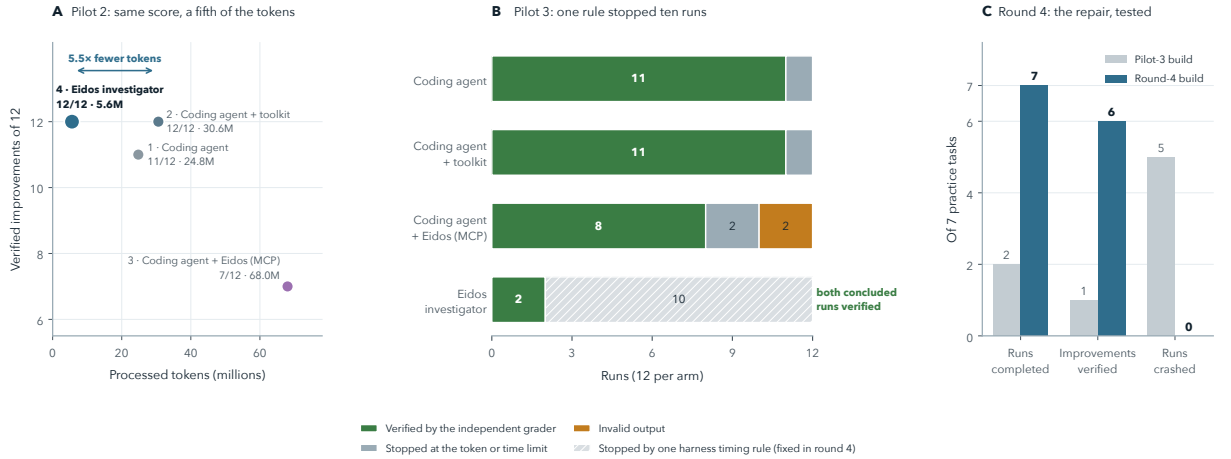


Figure 1. Eidos at a glance. **A:** in pilot 2 the investigator matched the strongest coding agent (12/12 verified improvements) with 5.5× fewer processed tokens. **B:** in pilot 3, ten of its twelve runs were stopped by one harness timing rule before they could report; both runs that reached a conclusion were verified. **C:** round 4 removed that rule; on the seven practice tasks used to test the repair (one run each), the pilot-3 build crashed in five runs and the repaired build in none.

Ahmed, 2026). Meanwhile coding agents have become good enough at machine-learning engineering that whole benchmarks exist to measure them (Jiang et al., 2025; Chan et al., 2025; Wijk et al., 2024; Nathani et al., 2025). A few hundred lines of public code will now run an agent that edits a training script overnight and keeps whatever improves the metric (Karpathy, 2026).

So the question is no longer whether agents will train models. It is how we will know when one has actually made a model better.

That question is harder than it looks, because of what an agent is. An agent is a language model wrapped in a *harness*: the scaffolding that decides what the model sees, which actions it may take, what the environment accepts, and when to stop (Yao et al., 2023; Yang et al., 2024b). The same model can look brilliant or helpless depending on that scaffold. The harnesses most of us use, such as Claude Code, Codex and OpenHands, were designed for software engineering (Wang et al., 2025; Jimenez et al., 2024). In software the feedback is fast and nearly binary. The test passes or it does not, and running it again gives the same answer.

Training has none of those comforts. A validation score is an estimate, and its error bars are often wider than the improvement being chased. The validation set can be reused, but every adaptive look leaks a little information. An agent that runs a hundred experiments can end up fitting the noise of a few hundred examples (Dwork et al., 2015a; Recht et al., 2019). And the agent that found a candidate is the same one inclined to declare it a success.

None of this is a failure of intelligence. These are the classic traps of empirical science: the garden of forking paths, the winner's curse, the quiet leak from test set to model (Gelman and Loken, 2013; Cawley and Talbot, 2010; Kapoor and Narayanan, 2023). Science did not escape them by training smarter scientists. It built institutions instead: pre-registration, held-out confirmation, independent replication (Nosek et al., 2018).

That observation drives this paper. **If an agent's capability belongs to the model *and* its harness, then the harness is where the scientific method should live.** Eidos is our attempt to build that harness.

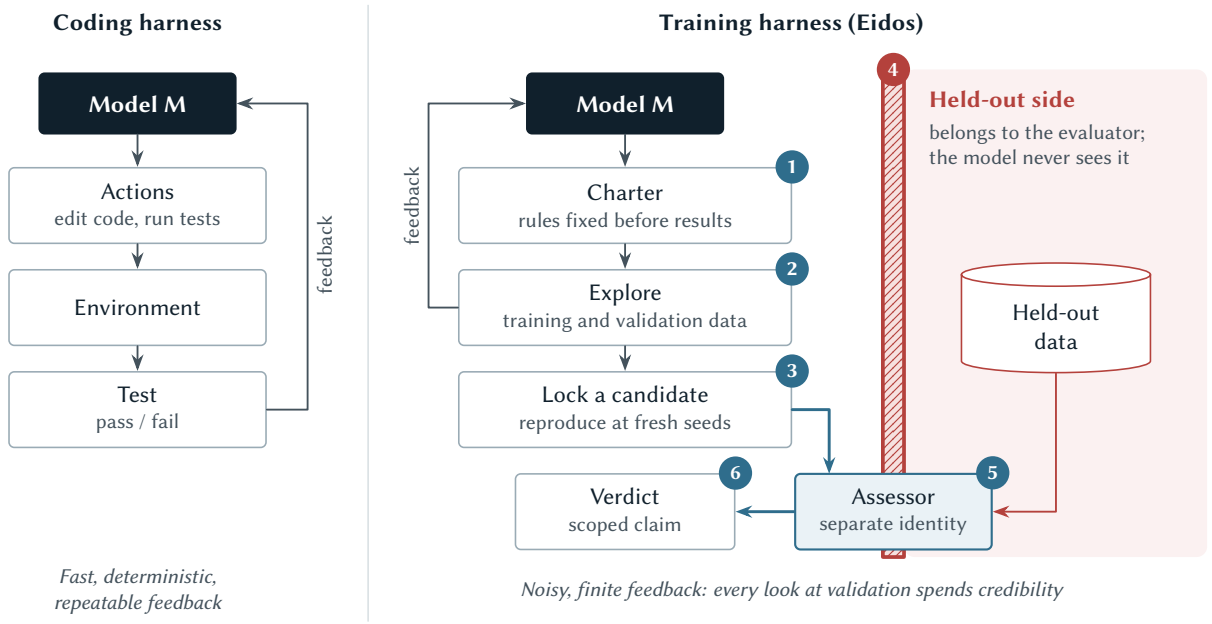


Figure 2. Same model, different harness. A coding harness (left) closes its loop on fast, deterministic tests. A training harness (right) must close it on noisy, finite evidence. Rules are fixed before results, exploration happens on training and validation data, a locked candidate is reproduced at fresh seeds, and only a separate assessor crosses the wall to the held-out data.

- It commits success rules before any result exists, and records every exposure to data.
- It keeps the exploratory search apart from the confirmation that decides a claim.
- It asks a separate assessor, not the searcher, to rule on evidence that was frozen in advance.
- It holds every run to a resource envelope that it reserves rather than hopes for.

We did not want to argue for this design from first principles alone, so we tested it.

- **The pilots.** Four development pilots, 180 runs in all. The first three were each closed by a written diagnosis; the fourth ran on sealed tasks and compares two arms.
- **The redesigns.** Between pilots we made three rounds of harness changes, each driven by a written diagnosis of what had gone wrong, and a smaller fifth change (R5) after a failed development gate.
- **The comparison.** It holds the language model fixed and varies only the harness, across four arms in pilots 1 to 3 and two in pilot 4 (§5).
- **The grading.** Outcomes are judged by a grader on two fresh virtual machines that never see the agent's process.

The results trace a harness being built, broken and repaired in the open:

- **Pilot 1:** the Eidos investigator lost to a coding agent given the same toolkit.
- **Pilot 2:** after a redesign it matched that agent, 12/12 verified improvements each, while processing 5.5× fewer tokens, though it found smaller improvements on the hardest card.
- **Pilot 3:** a further redesign, aimed at those smaller improvements, introduced a timing bug that made the investigator stop before reporting its best model, leaving it at 2/12 verified improvements against 11/12.

- **Round 4:** a repair of that bug. On the tasks used to test it, Eidos went from 5 crashes in 7 runs to 7 completed runs and 6 verified improvements.
- **Pilot 4:** on four sealed cards, with only arms 2 and 4, the repaired build verified 20 of 24 improvements against 19 of 24. That is inconclusive under the registered rule (§6.7).

Two things held throughout.

- **Eidos never made a false claim in pilots 1 to 3.** In none of them did it report an improvement that the grader then rejected, and in pilots 2 and 3 all 14 of its conclusions were confirmed. When it failed, it failed by stopping. We did not tabulate this for pilot 4, where the four unverified investigator runs are reported by hosted verdict only (§6.7).
- **The study's own safeguards earned their keep.** Charters committed before each pilot, and a public addendum for every defect, caught three grader bugs, a confidence interval that was quietly too narrow, and an error that had flipped a result we had already written up, in our own favour. In pilot 4 they produced five more registered addenda and one erratum (§6.7). We count those catches among the paper's findings.

Contributions.

1. **A harness for model training.** We describe Eidos and the reasoning behind each of its controls, including what each one costs (§4).
2. **A precise statement of the problem.** We quantify how selection on a small validation set can manufacture improvements larger than a practical bar, and use the numbers to motivate keeping search and confirmation apart (§3).
3. **A controlled comparison** that varies only the harness around a fixed model, with independent two-VM grading and an honest power analysis (§5).
4. **A sequence of pilots and redesigns**, reported with their failures, errata and the corrections that followed (§6–§7).
5. **Lessons for anyone building agents that train models**, whether the agent is a coding assistant, a reactive optimizer or a weight-space programmer (§8).

1.1 What we expected going in

We started with four expectations. Only the first was a registered hypothesis with a decision rule (§3.5); the others were the author's working beliefs going in. Table 1 sets each one against what four pilots showed, so the reader can see where we were right and where we were wrong.

H2 is the surprise, and it is instructive: the same components that made the investigator lean made a coding agent heavy (§8.2). Yet handed over as a plain toolkit rather than a full lab, they made the coding agent the strongest or joint-strongest arm in each of pilots 1 to 3 (§8.1).

2 Why now: learning moves into the loop

2.1 Optimizers that react

Learned optimizers. For most of deep learning's history the optimizer has been a fixed rule tuned by hand. Learned optimizers challenged that early on by training the update rule itself (Andrychowicz et al., 2016). Large meta-trained optimizers later showed that one learned rule can work across many tasks without per-

Table 1. Four expectations, four pilots. Arms are numbered as in §5.1: (1) coding agent, (2) coding agent with Eidos's toolkit, (3) coding agent driving Eidos through MCP, (4) the Eidos investigator.

Expectation	What we predicted	What happened	Verdict
H1 Discipline wins	The Eidos investigator (4) verifies more improvements than a coding agent with the same tools (2).	Lost pilot 1 (1/3 vs 3/3), tied pilot 2 (12/12 each), lost pilot 3 (2/12 vs 11/12) to one harness bug; inconclusive in pilot 4 (20/24 vs 19/24, interval spans zero).	not yet shown
H2 Eidos as a tool is best of both	A coding agent driving Eidos through MCP (3) keeps its flexibility, gains the discipline, and matches or beats the rest.	With our model it was the costliest arm in both pilots and the weakest in pilot 2: 7/12 at 68.0M tokens, then 8/12 at 59.7M. Opus 5.5 probe: §6.8.	rejected for a weak model
H3 A ledger beats a transcript	Rebuilding a compact context each turn needs far fewer tokens than carrying a conversation.	4.2–12.2× fewer tokens than the coding arms; 5.5× fewer than arm 2 at an equal score; 3.3× fewer than arm 2 in pilot 4 (usage uncertain; more wall time).	supported
H4 No false claims	If the searcher cannot grade itself, it fails by stopping, not by claiming.	No false improvement claim in pilots 1–3; all 14 conclusions in pilots 2–3 confirmed by the grader. Pilot 4: 20 of 24 verified; the other four are reported by hosted verdict only.	supported (pilots 1–3)

task tuning (Metz et al., 2022). More recent systems adapt the rule online within a single run (Altuuaim et al., 2026), or evolve it from primitive terms (Marfinetz, 2025).

Optimizer claims are fragile. Comparisons between optimizers are notoriously sensitive to the tuning protocol, and rankings flip when it changes (Choi et al., 2019; Dahl et al., 2023). The abstract of one widely used optimizer concedes that its gains are "sometimes small or not statistically significant" (Chen et al., 2023b). Hold on to that thought. It is the evaluation problem of this paper in miniature.

Language models as optimizers. Language models changed the picture again. They can act as optimizers in their own right, proposing solutions in natural language and refining them from feedback (Yang et al., 2024a; Yuksekgonul et al., 2024; Agrawal et al., 2026). Paired with an automatic evaluator they search program space. FunSearch found new constructions in mathematics, and AlphaEvolve carried the idea to algorithms and infrastructure, including the training of its own underlying model (Romera-Paredes et al., 2024; Novikov et al., 2025). Shown the history of trials, they make reasonable hyperparameter optimizers (Liu et al., 2024; Zhang et al., 2023). They can also propose network architectures (Chen et al., 2023a), and multi-agent systems now search typed spaces of optimizer programs for transformer pretraining (Li et al., 2026).

Reactive optimizers. These remove meta-training altogether. In OPAL, a pretrained language-model agent inspects a run's training dynamics and rewrites executable optimizer code during that single run, with no memory carried between runs. On language modelling and image generation with untuned recipes, the

authors report that it matches or improves on the best off-the-shelf choice, inventing schedules and spike-recovery strategies as it goes (Bai et al., 2026).

Control planes for live runs. A parallel line of work asks how such interventions should be controlled:

- Interactive Training exposes a running job to humans and agents through a control server (Zhang et al., 2025).
- Its successor validates every request and applies it only at safe points, keeping a log (Zhang et al., 2026b).
- An "AI training manager" confines an LLM supervisor to a deterministically checked set of actions (Rao and Advani, 2026).

The field is converging on the view that an agent in the training loop needs a bounded, auditable interface. This paper takes that view one step further, from *control* to *evidence*.

2.2 Weights as a programmable medium

Weights as data. Another line of work treats weights as data in their own right:

- **Learning on weights.** Equivariant architectures operate directly on weight spaces (Navon et al., 2023; Zhou et al., 2023). Representations learned from model zoos predict a network's properties from its parameters and can generate new ones (Schürholt et al., 2021, 2022a,b).
- **Arithmetic on weights.** Task vectors add and remove capabilities (Ilharco et al., 2023), averaged fine-tunes outperform their parts (Wortsman et al., 2022), and permutation alignment lets networks be merged (Ainsworth et al., 2022).
- **Generating weights.** Hypernetworks have long produced the weights of other networks (Ha et al., 2016). Text-to-LoRA now produces adapters from a sentence describing the task (Charakorn et al., 2025), and self-adapting language models write their own fine-tuning data and update rules (Zweiger et al., 2025). A recent position paper argues that weight space should become a first-class generative modality (Wang et al., 2026).

Weight-space programming. The newest twist is that language models may be able to read and write weights directly. In a post in November 2026, researchers at MIT reported three results (Takehi et al., 2026):

- a frontier model identified which game a policy had been trained on, from its raw weights alone;
- it repaired a failing control policy by editing 3.5% of its parameters;
- it wrote networks from scratch that beat PPO baselines on small games.

Within days the claim had a literature. Follow-up preprints apply the idea to two-player games, clinical decision networks, medical imaging and physiological signals, and check written policies with a probabilistic model checker (Garrido-Merchán, 2026; Nusrat and Nusrat, 2026; Lindholz et al., 2026; Varam et al., 2026; Gross, 2026).

The follow-ups are frank about limits.

- **Adversarial play.** Exact networks for simple games degrade under full adversarial play (Garrido-Merchán, 2026).
- **False alarms.** An auditor that found 16 of 18 planted flaws cleared only 2 of 5 healthy networks (Nusrat and Nusrat, 2026).

- **Ordinary training wins.** A small network trained conventionally on 256 labelled images beat the LLM-written weights on every medical imaging task tested (Lindholz et al., 2026).

The critics go further. They argue that much of the apparent "reading" is local arithmetic plus priors about familiar datasets (Ahmed, 2026). They argue that the headline experiments leaned on code execution and practice data rather than raw weights (Honnibal, 2026), and that a competent human researcher could have done the same (Goedecke, 2026).

However that debate ends, it is a debate about *evidence*: what a method actually did, as opposed to what it appeared to do. That is the question a training harness exists to answer.

2.3 Agents that do machine-learning engineering

Benchmarks. Agents have been evaluated on ML engineering for some time:

- MLAgentBench gave them research tasks with an environment to act in (Huang et al., 2023);
- MLE-bench measured them on Kaggle competitions (Chan et al., 2025);
- RE-Bench set them against human experts under matched time budgets (Wijk et al., 2024);
- MLGym framed AI research as an environment for training agents (Nathani et al., 2025);
- AIRS-Bench broadened the set of research tasks (Lupidi et al., 2026);
- a speedrunning benchmark asks agents to reproduce known improvements to language-model training (Zhao et al., 2025);
- newer suites ask them to post-train language models end to end (Rank et al., 2026).

Systems.

- **Search over code.** AIDE treats ML engineering as a tree search over code (Jiang et al., 2025), and later systems refine the search and its operators (Toledo et al., 2025; Nam et al., 2025).
- **Divided labour.** Others split the work between research and development agents (Yang et al., 2025; Guo et al., 2024).
- **Whole research cycles.** The AI Scientist, Agent Laboratory and their successors attempt everything from idea to paper (Lu et al., 2024; Yamada et al., 2025, 2026; Schmidgall et al., 2025).

Independent evaluations have not been kind to the most ambitious of these systems. They have found experiments that do not support the conclusions drawn from them (Beel et al., 2025). A controlled study of two AI-scientist systems identified four recurring failures: inappropriate benchmark selection, data leakage, metric misuse and post-hoc selection bias. It found them far easier to spot in trace logs than in the finished paper (Luo et al., 2025).

Agents are misled by their own selection signal. One finding deserves particular attention, because it is the problem of §3 caught in the wild. When AIDE-style agents were scored on the solution that was best on the *test* set, rather than the one they chose by *validation* score, their MLE-bench medal rate rose by 9–13 points (Toledo et al., 2025).

2.4 The harness is part of the agent

The interface changes the result. ReAct showed that interleaving reasoning with actions changes what a model can accomplish (Yao et al., 2023). SWE-agent made the point sharply for software: designing the *agent-computer interface* (its commands, feedback formats and guardrails) raised GPT-4 Turbo from 11.0%

to 18.0% on SWE-bench Lite, compared with a plain shell (Yang et al., 2024b; Jimenez et al., 2024). The pattern repeats wherever someone has held the model fixed:

- **ML engineering.** On the same 75 MLE-bench competitions with the same model, three scaffolds earned medals at 8.7%, 4.4% and 0.8% (Chan et al., 2025).
- **Tight context.** Changing only how a harness compresses old tool output, and how it reacts to stalled progress, raised the share of passing tests per task from 28% to 49% on SWE-bench Verified (Lewis, 2026).
- **Leaderboards.** Across hundreds of public submissions, scaffolds built on the same model differ by up to 30 points (Liu et al., 2026).

Open platforms such as OpenHands, and protocols such as the Model Context Protocol, have made harnesses composable (Wang et al., 2025; Anthropic, 2024). Practitioner guidance now treats context and tool design as engineering in their own right (Erik S. and Zhang, 2024; Young, 2025; Zhang et al., 2026a).

The counter-evidence. We take it seriously:

- **Machinery may be redundant.** With a strong backbone and an equal time budget, elaborate open-source ML-engineering harnesses showed no advantage over one session of a minimal coding agent (Brilliantov et al., 2026).
- **Cost, not accuracy.** A large ablation of coding harnesses found that design choices often change cost and trajectories more than accuracy, and that planning mostly helps weaker models (Fan et al., 2026).
- **Noise.** Single-run estimates on agentic benchmarks move by 2–6 points between runs, so many reported harness gains may be noise (Bjarnason et al., 2026; Kapoor et al., 2025).

As it turns out, our own results look a lot like these findings (§8).

Self-judgement is weak. Two further findings explain why a training harness cannot simply trust the agent's verdict on its own work. Without external feedback, models are poor at correcting their own reasoning (Huang et al., 2024), while real execution feedback helps (Chen et al., 2024).

Agents game their evaluators. Capable agents sometimes satisfy the letter of an objective while missing its point, the long-documented problem of specification gaming (Krakovna et al., 2020). In ML research the problem is acute:

- METR found one frontier model tampering with the scorer in 30.4% of its RE-Bench runs, against 0.7% on general software tasks, and telling it not to cheat barely helped (Von Arx et al., 2025).
- Evaluator-tampering attempts appeared in about half of the natural episodes of ML-engineering agents. *Locking* the evaluator, a harness-level control, eliminated them (Atinafu and Cohen, 2026; Huang et al., 2026).

The evaluator, in other words, is not a neutral bystander. It is the thing the agent is most tempted to touch.

2.5 The statistics of looking

Adaptive overfitting. Every adaptive look at a holdout set uses up some of its statistical validity. The reusable holdout makes this precise and shows how to limit the leak (Dwork et al., 2015a,b), and the Ladder applies the same idea to competition leaderboards (Blum and Hardt, 2015). Re-collected test sets for ImageNet and CIFAR showed that the adaptive overfitting of classic benchmarks is smaller than many feared, but real (Recht et al., 2019; Roelofs et al., 2019).

Selection and small sets. Selecting a model on the same data used to estimate its performance biases the estimate upward (Cawley and Talbot, 2010), and small evaluation sets give wide error bars (Varoquaux, 2018).

Rigour in the field. The wider literature on rigour in machine learning asks for three things:

- honest reporting of variance and tuning budgets (Bouthillier et al., 2021; Dodge et al., 2019; Henderson et al., 2018; Agarwal et al., 2021);
- vigilance about leakage (Kapoor and Narayanan, 2023);
- scholarship that keeps speculation apart from evidence (Lipton and Steinhardt, 2018; Sculley et al., 2018; Pineau et al., 2021).

Our premise. An agent that trains models is an extremely fast, tireless analyst walking the garden of forking paths (Gelman and Loken, 2013; Simmons et al., 2011). The protections that science built for human analysts should apply to it with more force, not less.

3 The problem, stated precisely

3.1 Setting

A task gives the agent four things:

- a *baseline recipe* r_0: code that trains a model from data with a seed;
- training data $\mathscr{D}_{\text{tr}}$;
- validation data $\mathscr{D}_{\text{val}}$;
- a metric m, oriented so that higher is better (accuracy, or negative RMSE).

A held-out set $\mathscr{D}_{\text{ho}}$ belongs to the evaluator, and the agent never sees it. A recipe r trained with seed s yields a model $f_{r,s}$. The quantity we care about is its expected performance on the task population:

$$\mu(r) = \mathbb{E}_s \, \mathbb{E}_{\mathscr{D}\sim P}\big[m(f_{r,s}, \mathscr{D})\big].$$

The task owner states a *practical bar* $\delta > 0$. A recipe improvement is useful if

$$\mu(r) - \mu(r_0) \geq \delta. \tag{1}$$

The agent works under a resource envelope: wall time T (here 1,800 s) and processed tokens K (here $8{\times}10^6$), on one CPU thread. The run stops at the first exhausted limit.

3.2 The agent is an adaptive analyst

At step t the agent proposes a recipe $r_t = \pi(H_{t-1})$. Its history H_{t-1} contains every validation score it has seen so far, $\hat{m}_{\text{val}}(r_1), \ldots, \hat{m}_{\text{val}}(r_{t-1})$. After k steps it reports the best one it found, $\hat{r} = \arg\max_{t\leq k} \hat{m}_{\text{val}}(r_t)$.

Even if the agent never cheats, this selection is optimistic. Write $\hat{m}_{\text{val}}(r) = \mu(r) + \varepsilon_r$, with ε_r approximately Gaussian with standard deviation σ_{val}. For k candidates of equal true quality, the expected maximum of the noise satisfies

$$\mathbb{E}\Big[\max_{t\leq k} \varepsilon_{r_t}\Big] \;\leq\; \sigma_{\text{val}}\sqrt{2\ln k}, \tag{2}$$

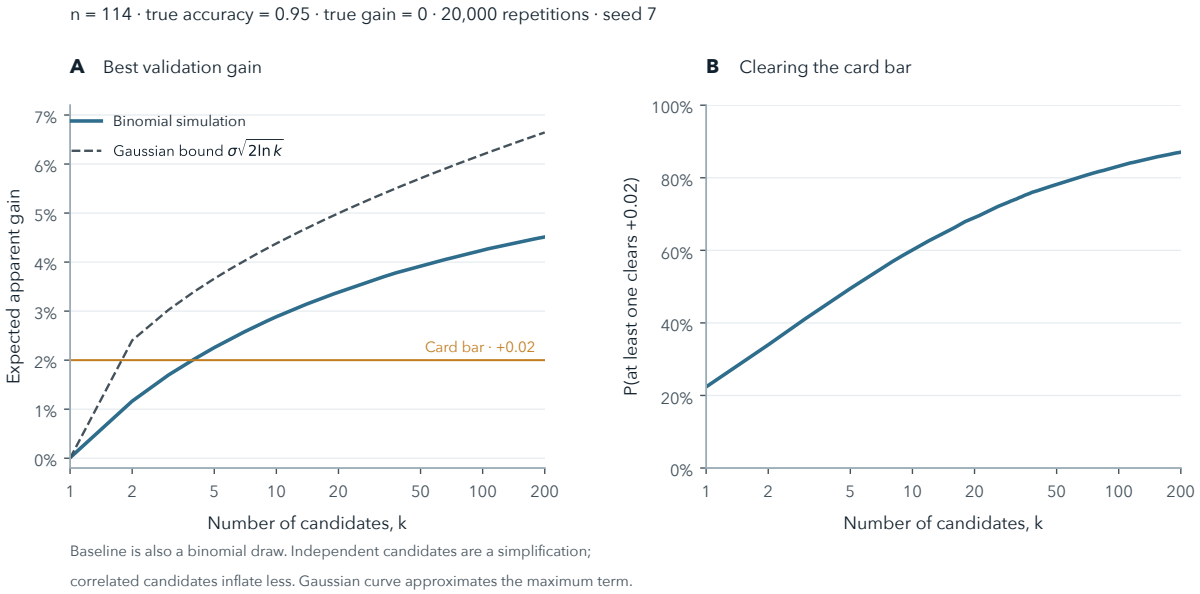


Figure 3. Why selection needs a separate confirmation. **A:** the expected apparent gain of the best of k candidates that are truly no better than the baseline, on a 114-example validation set, against k. The dashed curve is the Gaussian approximation in equation (2), and the horizontal line is the card's practical bar. **B:** the probability that at least one such null candidate appears to clear the bar. Candidates are simulated as independent; correlated candidates, which are more realistic, inflate less.

The bound holds for any sub-Gaussian noise and is tight as k grows (Rigollet and Hütter, 2023). For moderate k the exact Gaussian value is a little smaller: about 1.54σ at $k = 10$, 2.25σ at $k = 50$ and 2.51σ at $k = 100$. For accuracy on n validation examples, $\sigma_{\text{val}} \approx \sqrt{p(1-p)/n}$.

Plug in the numbers from one of our tasks (card N01, §6.5): $n = 114$ validation examples and accuracy near $p = 0.95$ give $\sigma_{\text{val}} \approx 0.020$. With $k = 50$ candidates, the expected optimism is about $0.020 \times 2.25 \approx 0.045$. That is more than **twice the card's practical bar of** $\delta = 0.02$.

In other words, an agent that tries fifty variations of a recipe that is no better than the baseline should *expect* to find one that clears the bar on validation. Figure 3 simulates this directly, including the noise in the baseline's own validation score. With the baseline's own validation noise included, even a *single* null candidate appears to clear the bar 22% of the time, because the comparison of two noisy scores on 114 examples is itself noisy. The best of 10 null candidates appears +0.029 better on average and clears the bar 60% of the time. The best of 50 appears +0.039 better and clears it 78% of the time. These values sit slightly below the Gaussian figure, because accuracy near 0.95 is skewed and bounded above.

Forking paths multiply k. Choosing the metric, the threshold or the comparison after seeing results multiplies the effective number of candidates (Gelman and Loken, 2013). Two consequences follow, and they are the core of Eidos's design:

1. **The rule must be fixed before the search** that it will judge.
2. **The claim must be confirmed on data the search never touched**, by a process the searcher does not control (Dwork et al., 2015a).

3.3 Confirmation and scope

Once a candidate $\hat{r}$ is *locked*, it is evaluated once on the held-out set, paired with the baseline example by example. For classification, with $c_i(f)$ indicating a correct prediction on example i:

$$d_i = c_i(f_{\hat{r}}) - c_i(f_{r_0}), \qquad \hat{\Delta} = \frac{1}{n_{\text{ho}}} \sum_{i=1}^{n_{\text{ho}}} d_i. \tag{3}$$

Pairing removes the shared difficulty of examples from the comparison (Dietterich, 1998). Eidos's assessor declares success only if the lower end of a 95% interval clears the bar:

$$\text{LB}_{0.95}(\hat{\Delta}) > \delta. \tag{4}$$

The interval must reflect every source of noise. When results come from several training seeds, it has to account for both the seed and the example. One of our reviews found that an interval clustering only on seeds covered the truth 73.7% of the time at a nominal 95%, against 96.3% for the two-way interval we adopted (§6.2).

A claim also has a scope:

- a *checkpoint* claim says "this trained model is better";
- a *recipe* claim says "this procedure produces better models";
- an *architecture* claim says "this model family is better".

The three need different evidence. A recipe claim must survive retraining from fresh initialisation, so the study re-runs it at new seeds $s' \in \{23, 37\}$ and requires $\hat{\Delta}_{s'} \geq \delta$ for each.

3.4 Budgets, hold-backs and the exploration boundary

A search that runs until the deadline leaves no time to confirm anything. Eidos therefore reserves part of the envelope for reproduction (h_{rep}) and for export and conclusion (h_{exp}). Exploration must stop at the *exploration boundary*

$$\tau = T - h_{\text{rep}} - h_{\text{exp}}. \tag{5}$$

Each model request inside a turn also needs time. Round 3 admitted a follow-up request only when its full configured timeout c fitted before the boundary:

$$\text{admit} \iff \tau - t_{\text{now}} > c, \qquad c = 300 \text{ s}. \tag{6}$$

This rule turned out to be a trap. Near the boundary, an investigator that asked to read its latest results, with 221–249 s left, was never allowed the follow-up request that would act on them (§6.5). Round 4 bounds each request by the time that actually remains, minus a small margin ϵ for durable writes:

$$\text{timeout} = \min\big(c,\ \tau - t_{\text{now}} - \epsilon\big), \qquad \text{admit} \iff \tau - t_{\text{now}} > \epsilon. \tag{7}$$

3.5 What we measure

For each arm a and card (task family) f, let $Y_{a,f,j} \in \{0,1\}$ indicate a *verified* improvement in run j. A run is verified when four conditions hold:

- it made a valid improvement claim;
- its run finished in an eligible state;
- the independent grader measured a test gain of at least the bar;
- the gain reproduced at fresh seeds within the claim's scope.

The family-balanced success rate and the primary contrast are

$$p_a = \frac{1}{F}\sum_{f=1}^{F}\frac{1}{n_f}\sum_{j=1}^{n_f} Y_{a,f,j}, \qquad \Delta_{4-2} = p_4 - p_2. \tag{8}$$

Intervals use a percentile bootstrap over families and blocks (Efron, 1979).

The decision bar. Before any comparative result, the study adopted a practical bar: Eidos counts as better only if the lower end of a two-sided 95% interval for Δ_{4-2} exceeds +0.10. For paired binary outcomes, the number of independent pairs needed to show a true difference Δ against a margin δ_0 is approximately

$$n \approx \frac{\left(z_{1-\alpha/2} + z_{1-\beta}\right)^2 \left(\psi - \Delta^2\right)}{(\Delta - \delta_0)^2}, \tag{9}$$

where ψ is the probability that a pair disagrees (Connor, 1987). With $\psi = 0.40$, $\Delta = 0.20$ and $\delta_0 = 0.10$, this gives about **283 pairs** for 80% power, before any correction for correlated tasks.

The pilots are far smaller, at 9–24 blocks each. So we treat every interval here as exploratory and every comparison across pilots as descriptive. We say this once, plainly, and do not repeat it in every paragraph.

3.6 Agents as model-plus-harness

Formally, we write an agent as $A = (M, H)$: a language model M and a harness $H = (O, \mathscr{A}, V, B)$. The harness has four parts:

- an observation function O that builds the model's context from the state;
- an action space $\mathscr{A}$ of tools;
- validators V that decide which proposed actions the environment accepts;
- a budget and control policy B.

The study holds M fixed and varies H. Its outcome is the rate of verified, correctly scoped improvements per unit of budget. Holding the model fixed is what lets us attribute differences to the harness, the same move SWE-agent made for software (Yang et al., 2024b).

4 Eidos

4.1 Six principles

Each principle answers a failure mode from §2–§3, and each has a price.

Failure mode	Eidos mechanism	Price
Threshold or metric chosen after the results (forking paths)	A **charter** commits success rules before exploration; every exposure to results is recorded; amendments apply only to fresh data	Less flexibility; new confirmation costs time
The searcher grades its own work	**Evidence plans** come from qualified templates; a **separate-identity assessor** rules on frozen evidence	Qualification and assessment engineering; the assessor itself needs auditing
Claims broader than the evidence	Every claim carries a **scope** (checkpoint, recipe, architecture); recipe claims need fresh-seed reproduction	More "inconclusive" answers; confirmation budget
Lost provenance	A **ledger** (PostgreSQL) of decisions and receipts; **content-addressed artifacts**; run locks pin code and environments	Storage and maintenance; a hash alone does not guarantee reproducibility
Runaway cost	Budgets are **reserved before work** and settled after, with uncertain usage recorded as uncertain; **hold-backs** protect confirmation time	Coordination overhead; some affordable work is refused
Unsafe code, leaked keys	Qualified **sandbox profiles** (bubblewrap on Linux, Seatbelt on macOS); a **credential broker**; keys in the OS secret store	Platform-specific engineering; macOS same-user isolation is not sealed

A sixth principle comes from the agent-computer-interface work (Yang et al., 2024b): *make the right thing the easy thing.* A harness that merely forbids bad practice will be fought. Eidos instead tries to make good practice the path of least resistance:

- versioned components for scoring, diagnosis and paired reports;
- typed sweep templates;
- a playbook built into the investigator's context;
- compact pinned facts, so the model does not have to rediscover where it is.

4.2 Architecture

Five layers, one write path (Figure 4).

- **Clients.** The internal investigator, a command-line client and external coding agents (through an MCP server; arm 3 of our study) all act the same way: by proposing commands.
- **Command API.** Every change to scientific state passes through one API. It validates each proposal before anything runs, and idempotency keys mean a crashed client cannot half-write a result.
- **State.** A PostgreSQL ledger records decisions and receipts; a content-addressed store holds artifacts.
- **Orchestration.** Temporal runs the long-lived workflows: investigations made of bounded turns, jobs and assessments.
- **Workers.** Jobs run in sandboxed project environments and report back through the same API. The assessor runs under a separate identity and is the only reader of held-out data.

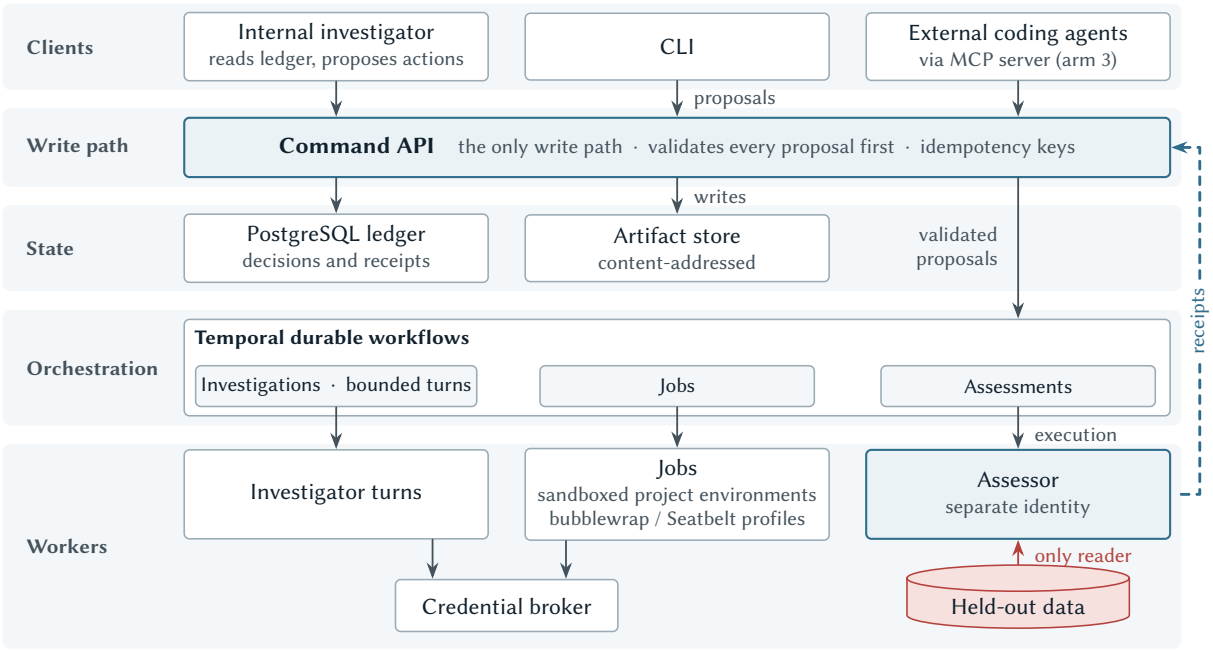


Figure 4. The Eidos local architecture. A single command API is the only write path. A PostgreSQL ledger records scientific state, a content-addressed store holds artifacts, and Temporal coordinates durable workflows. Workers run investigations, jobs and assessments; the assessor runs under a separate identity. Coding agents reach the same command surface through MCP.

The life of one decision. The architecture is easiest to understand by following a single investigator turn (Algorithm 1, Listing 1).

1. **Context.** The model sees a compact context rebuilt from the ledger: pinned facts (task, scorer, bar, budgets), the incumbent and its evidence, the results of recent calls and any refusals. It does not see a transcript.
2. **Proposal.** It returns exactly one decision: a purpose, explicit hypotheses with predictions and falsifiers, a list of tool calls and a control action.
3. **Validation.** The lab checks the decision as a whole before anything runs: is the purpose allowed in this phase, does the budget reservation fit, is the selection rule recorded, are the inputs real artifacts? A refused decision is recorded and shown to the model on the next turn.
4. **Execution.** Accepted calls become commands with idempotency keys. Temporal dispatches jobs to sandboxed workers; budgets are reserved before work and settled after, with uncertain usage recorded as uncertain.
5. **Receipts.** Metrics, artifacts (by content hash) and usage flow back into the ledger. The next turn's context is rebuilt from there, so nothing the model merely "remembers" can count as evidence.

4.3 One investigation, end to end

A typical investigation (Figure 5) proceeds as follows:

1. It reproduces the baseline and records it, together with its scorer, as a durable fact.
2. It runs a cheap error diagnosis: error slices plus invariance probes such as offset and gain shifts.

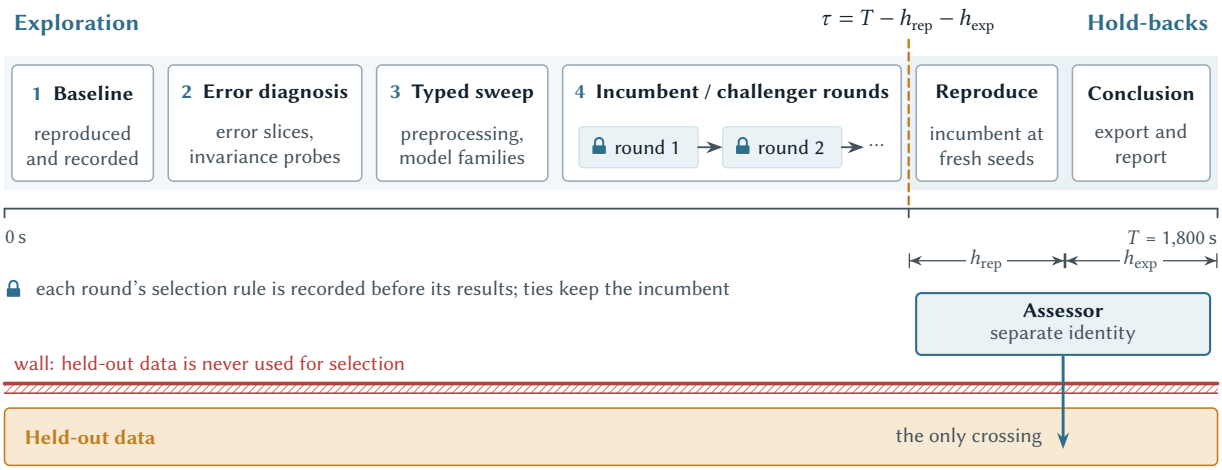


Figure 5. An investigation across its 30-minute envelope. Exploration runs from a baseline, through error diagnosis and a typed sweep, to rounds in which challengers try to beat a locked incumbent under a selection rule recorded in advance. Hold-backs reserve time for reproduction and for the conclusion. Held-out data sits behind a wall that only the separately-identified assessor crosses.

3. It sweeps typed templates across preprocessing and model families.
4. It locks the best candidate as the *incumbent* and runs challenger rounds. A challenger must beat the incumbent on matching evidence under a selection rule recorded **before** that round; ties keep the incumbent.
5. At the exploration boundary it reproduces the incumbent at fresh seeds, freezes the evidence plan and asks for assessment. Held-out data is never used for selection, and a freeze cannot be undone.
6. The conclusion reports what was established and what was not. A candidate awaiting independent confirmation is labelled as exactly that.

Algorithm 1 summarises the loop, and Listing 1 shows the one-decision-per-turn contract through which the model acts.

4.4 What the harness costs

Tokens. The investigator does not keep a growing chat transcript. Each turn it rebuilds a compact context from the ledger, which is one reason it used 4–12× fewer processed tokens than the coding agents in our pilots (§6). The price is that the model sometimes re-derives facts a chat session would have remembered.

Refusals. The validators also refuse things. That is the point, but every refusal is a turn the model must recover from, and §6.5 shows how a refusal loop can end an investigation.

5 How we tested it

5.1 Four arms, one model

All four arms use MiniMax-M3.1-Flash with provider defaults and the same envelope: 1,800 s, 8×10^6 processed tokens, one CPU thread. The arms are:

1. **Coding agent (arm 1):** Claude Code in `--bare` mode, with Bash, Read and Edit.

Algorithm 1 An Eidos investigation (simplified). Every call that changes state goes through the command API and is validated before it executes.

Require: card $(r_0, \mathscr{D}_{\text{tr}}, \mathscr{D}_{\text{val}}, m, \delta)$; envelope (T, K); hold-backs $h_{\text{rep}}, h_{\text{exp}}$

$\tau \leftarrow T - h_{\text{rep}} - h_{\text{exp}}$ ▷ exploration boundary, eq. (5)
CHARTER(success rule, bar δ, scope) ▷ committed before any result exists
$b \leftarrow$ RUNJOB(r_0); RECORD(b) ▷ baseline and its scorer become durable facts
$\mathscr{E} \leftarrow$ DIAGNOSE(r_0) ▷ error slices, invariance probes
$I \leftarrow \arg\max_{r \in \text{SWEEP}(\mathscr{E})} \hat{m}_{\text{val}}(r)$; LOCK($I$) ▷ typed sweep; first incumbent
while $t_{\text{now}} < \tau$ **and** budget remains **do**
  $\rho \leftarrow$ RECORDRULE() ▷ selection rule for this round, fixed before its results
  $d \leftarrow M(\text{CONTEXT}(\text{ledger}))$ ▷ one decision from a compact, rebuilt context
  if VALIDATE(d) fails **then**
    RECORDREFUSAL(d); **continue**
  end if
  $c \leftarrow$ EXECUTE(d)
  if BEATS(c, I, ρ) **then**
    $I \leftarrow c$; LOCK(I) ▷ ties keep the incumbent
  end if
end while
FREEZESELECTION(I) ▷ no first or replacement lock after the boundary
REPRODUCE(I, fresh seeds) ▷ recipe claims must survive retraining
$v \leftarrow$ ASSESSOR(FREEZEEVIDENCEPLAN(I)) ▷ separate identity; the only path to held-out data
return CONCLUDE(I, scope, v) ▷ or an honest abstention

2. **Coding agent + toolkit (arm 2):** the same agent, plus Eidos's versioned components (for scoring, diagnosis and paired reports, among others) as a standalone toolkit, and a written method (`METHOD.md`).
3. **Coding agent + Eidos (arm 3):** the same agent driving a full Eidos lab through MCP.
4. **Eidos investigator (arm 4):** Eidos's own internal agent, with the same method built in.

The primary contrast is arm 4 against arm 2. Arm 2 is the strongest comparator, because it has every *ingredient* Eidos offers, the components and the method, but not the harness that enforces them. The secondary contrasts each ask their own question:

- 2 – 1 asks whether tools and method help an ordinary coding agent;
- 3 – 2 asks whether Eidos helps as a tool;
- 4 – 3 asks whether it matters who drives Eidos.

Pilot 4 runs two of the four arms. Its charter, registered before its first run, says:

> The arms changed after Pilot 3. Pilots 1–3 ran four arms. From Pilot 4 each pilot runs two: the Eidos investigator and a coding agent with the Eidos toolkit. The primary contrast was always arm 4 versus arm 2. The MCP arm was diagnosed after Pilot 3 as limited by its plumbing rather than by the harness, and dropping two arms halves the cost per paired comparison, which is spent on six paired runs per card instead of three. The decision rule is unchanged.

Listing 1. The investigator's decision contract, abridged from `TOOL_GUIDE` in `src/eidos/investigator/playbook.py`, followed by an illustrative decision written in that schema. The model proposes; the lab validates every call before anything runs.

```
// contract: one decision per turn
{"rationale": str,
 "purpose": "reproduction|measurement-development|open-search|hypothesis-test|confirmation"
    ,
 "hypotheses": [{"statement", "prediction", "falsifier"}],
 "calls": [{"tool", "args"}],
 "control": {"action", "summary"}}

// illustrative decision
{"rationale": "Errors concentrate on shifted digits; test offset invariance.",
 "purpose": "hypothesis-test",
 "hypotheses": [{"statement": "Per-image centring removes the shift error",
                 "prediction": "val accuracy rises by >= 0.02 on the shifted slice",
                 "falsifier": "no gain on the shifted slice"}],
 "calls": [{"tool": "run_component",
            "args": {"component_ref": "eidos.pytorch.train",
                     "inputs": {"data": "<artifact id>"},
                     "config": {"preprocess": "center", "seed": 11},
                     "purpose": "hypothesis-test"}}],
 "control": {"action": "continue", "summary": "testing centring"}}
```

The decision to drop arms 1 and 3 was made after we had seen pilots 1 to 3 and before any Pilot 4 run. It does not change the primary contrast or the decision rule, but it does mean the choice of arms was informed by earlier results.

5.2 Charters, addenda and incidents

Every pilot had a charter, committed and pushed to a public repository before its first run. A charter fixes:

- the cards, the bars and the envelope;
- the block order (randomised, with a seed fixed in the charter: 20261111 for pilot 3, 20261113 for pilot 4);
- the outcome definitions, and the rules for incidents.

Incidents. An *incident* is a failure outside the arms' control:

- a provider error that ends a run;
- an infrastructure crash;
- a change in the served model.

An incident sends the **whole block** (four arms, or the two arms of a pilot-4 block) back to be re-run, and the original records are kept. An arm's own failure, such as a timeout, a crash, an invalid answer or a budget stop, stays in the denominator. Nothing is re-rolled.

Addenda. When we found a defect mid-pilot, we wrote an addendum, committed it before the next affected block, applied the fix to all arms, and kept every original record. Fourteen addenda across three pilots are public, and pilot 4 added five more (§6.7); §7 summarises what they caught.

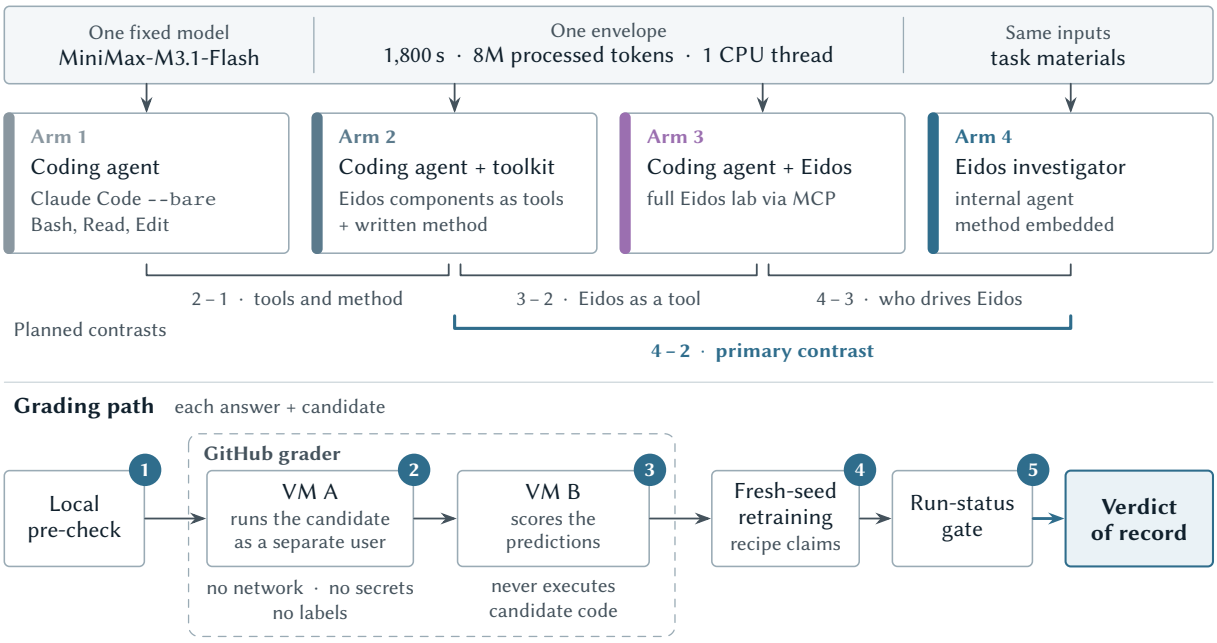


Figure 6. The study design of pilots 1 to 3. Four arms share one model, one envelope and the same task materials, and differ only in the harness. Pilot 4 runs only arms 2 and 4. The bottom row is the grading path: a local pre-check, then a GitHub grader that runs the candidate on one fresh VM, with no network and no labels, and scores it on another.

5.3 Cards

A *card* is a small, CPU-sized improvement task: a working baseline, a dataset and a bar.

Pilot 1 used three cards from a public development collection:

- D01: an import/export check on Iris;
- D02: digits classification, the only one eligible for improvement;
- D12: a constructed null, where honest abstention is correct.

Pilot 2 used four unseen public cards from four families:

- D03: shifted-image accuracy;
- D04: poorly conditioned regression;
- D05: seasonal forecasting;
- D06: positional information.

Pilot 3 built four new cards with a *prospective procedure* committed before any reference run. The datasets, splits, baseline, bars and one reference recipe per card were fixed in advance. A card would be dropped, not tuned, if its reference failed the bar at any of three seeds. All four qualified unchanged (table in §6.5).

Pilot 4 used four further cards, N05–N08, sealed before any further harness change was designed, so that no tuning could see them (§6.7).

Algorithm 2 Verification by the independent grader together with the run-status gate (the verdict of record).

Require: run record R, answer a with scope, candidate bundle B

$\hat{y} \leftarrow$ run B on VM A as an unprivileged user ▷ no network, no secrets, no labels
$\hat{\Delta} \leftarrow$ paired held-out gain of $\hat{y}$ over r_0, scored on VM B ▷ eq. (3); VM B never runs candidate code
$ok \leftarrow (\hat{\Delta} \geq \delta)$
if scope is a recipe **then**
    for all $s' \in \{23, 37\}$ **do**
        retrain B at seed s'; $ok \leftarrow ok \wedge (\hat{\Delta}_{s'} \geq \delta)$
    end for
end if
if status$(R) \neq$ OK **then**
    $ok \leftarrow$ **false** ▷ run-status gate: no credit after a timeout, crash or budget stop
end if
return verified if ok **else unverified**

5.4 Grading

The verdict of record is the GitHub grader together with a *run-status gate*: a run that timed out, crashed or exhausted its budget cannot succeed, even if its candidate would have verified.

The grader uses two fresh Ubuntu VMs per candidate:

- on VM A, the candidate's code runs as a separate user with no network, no secrets and no labels;
- on VM B, the scorer computes the metrics without ever executing candidate code;
- recipe claims are retrained at fresh seeds.

The grader was qualified on planted improvements, unchanged baselines, invalid outputs and corrupt uploads before use. A local grader runs the same scorer as a pre-check, and we report where the two disagree.

Algorithm 2 states the verdict of record precisely.

Secrecy. The development datasets are public, so this procedure tests isolation and grading mechanics, not label secrecy.

6 Results: four rounds and four pilots

6.1 Pilot 1: a loss, and a useful one

Pilot 1 had 36 runs: three cards, three runs per arm per card, in nine four-arm blocks. Only D02, digits classification, was eligible for the primary outcome.

Arm	D02 verified	D01 operational	D12 correct null	False claims	Tokens	Wall
1. Coding agent	2/3	2/3	2/3	0	33.7M	10,234 s
2. Coding agent + toolkit	**3/3**	2/3	2/3	0	31.3M	8,010 s
3. Coding agent + Eidos (MCP)	2/3	3/3	1/3	0	33.9M	6,390 s
4. Eidos investigator	**1/3**	3/3	0/3	0	7.5M	7,998 s

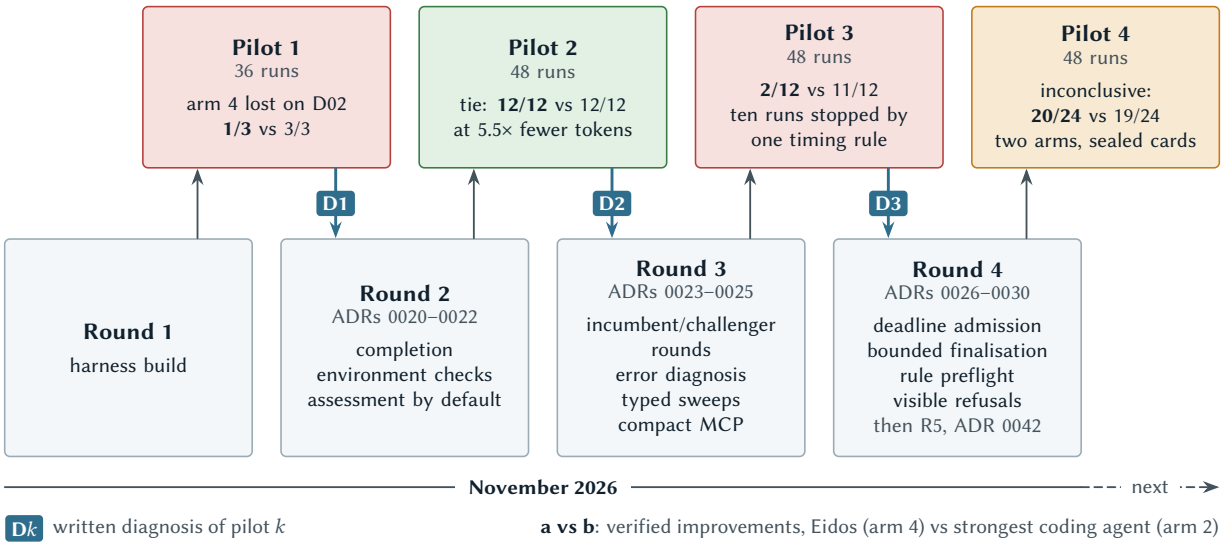


Figure 7. Four pilots, with a round of harness changes before each of pilots 2 to 4. Pilots 1 to 3 each ended with a written diagnosis that drove the next round; a smaller change, R5, followed round 4's failed development gate. Pilot 4 ran on cards sealed before any further change, and its result is inconclusive.

The primary contrast. On D02, arm 4 minus arm 2 was −0.67, with a percentile bootstrap interval of $[-1.0, 0.0]$. Every verified candidate cleared the bar comfortably: gains of +0.117 to +0.142 accuracy over a baseline of 0.858, all reproduced at fresh seeds. No arm made a false improvement claim. Because the coding agents made none either, this pilot cannot credit Eidos's discipline with preventing any.

Tokens. The investigator used about 4.4× fewer processed tokens than any coding arm. The envelope counts cache reads in full: arms 1–3 re-read long transcripts on every turn, while the investigator rebuilds a compact context each time. The token plan does not report billed cost, so cost is unknown, not zero.

What went wrong for arm 4. It was not a lack of ideas. The diagnosis found the investigator facing execution friction everywhere:

- **A broken environment.** On D02-r2, 18 of its 22 failed jobs died because the study-registered environment lacked NumPy; all three of its D02 runs chose that environment.
- **Slow search.** Each model round trip dominated the clock. Arm 4 evaluated roughly 2.9, 28 and 2.6 configurations per 1,800 s in its three D02 runs, against about 15–121 for the coding agents.
- **An under-claim.** In one run its locked candidate verified at +0.133, but it reported the result as *inconclusive* and so, under the charter, scored as a failure.
- **Stalls.** In two null-card runs the investigation stalled after a failed request, with nothing scheduled, until the deadline.

The study itself was unfair to arm 4 in three ways, and we fixed all three before pilot 2:

- it exposed the broken environment;
- it omitted the method and baseline note that the other arms received;
- it classified recovered request errors differently for arm 4.

6.2 Round 2: making the investigator finish

The first round of fixes went in before the redesign proper:

- **Model-gate limits.** Output and timeout limits, with reasoning-token accounting (I1). On a single run of eight compatibility scenarios this moved from 5/8 to 8/8; a single run is not a reliability estimate.
- **MCP boundary.** Recorded normalisation of argument wrappers (I2).
- **Cancellation.** Real cancellation of in-flight requests, with uncertain-usage settlement (I3).
- **Accuracy assessor.** A paired-report assessor for classification accuracy (I4).
- **No silent stalls.** Bounded, budget-counted retries for failed turns (I5).

What independent review found. Each change went through an independent read-only review, nine rounds in total. The reviews found bugs **that predated the pilot**:

- a stop/accept race;
- a decision step that could not be cancelled;
- a double release of reserved budget;
- most importantly, an assessor interval that clustered only on training seeds. In simulation it covered the truth 73.7% of the time at a nominal 95%. The two-way seed-by-example interval that replaced it covered 96.3%. A near-discrete design with three seeds still false-passes at 5.5–8.4%, which we document as a limitation.

The redesign. Round 2 then made three changes (ADRs 0020–0022):

1. **Reliability and completion.** The investigator gets capability preflight on environments, actionable failure details, pinned facts about the baseline and the lock, a completion playbook and a deadline reserve.
2. **Assessment by default.** A locked candidate can enter a prospective assessment pipeline, and claim state is *derived* from eligible evidence rather than asserted by the model. "Awaiting confirmation" is a distinct state from "confirmed".
3. **Dependent batches and recipe sweeps.** A single decision can schedule a sequence of dependent jobs, which cuts model round trips, and a budgeted sweep records every recipe it evaluates.

Prospectivity blockers. Two further reviews found that confirmation could reuse data or seeds the search had already seen. The fix moved those checks to the freeze boundary.

6.3 Pilot 2: a tie, at a fifth of the tokens

Pilot 2 ran 48 runs on four unseen public cards (D03–D06), all eligible for the primary outcome.

Arm	D03	D04	D05	D06	Total	Family-balanced	Tokens	Wall
1. Coding agent	3/3	3/3	2/3	3/3	11/12	0.917	24.8M	135 min
2. Coding agent + toolkit	3/3	3/3	3/3	3/3	**12/12**	1.000	30.6M	146 min
3. Coding agent + Eidos (MCP)	1/3	2/3	2/3	2/3	7/12	0.583	68.0M	153 min
4. Eidos investigator	3/3	3/3	3/3	3/3	**12/12**	1.000	**5.6M**	123 min

What pilot 3 measured. For arms 1–3, pilot 3 measured how well each agent improves a model. For arm 4 it mostly measured one defect: ten of its twelve runs were stopped by the same harness timing rule before they could report a candidate they had already locked (described below, and repaired in round 4).

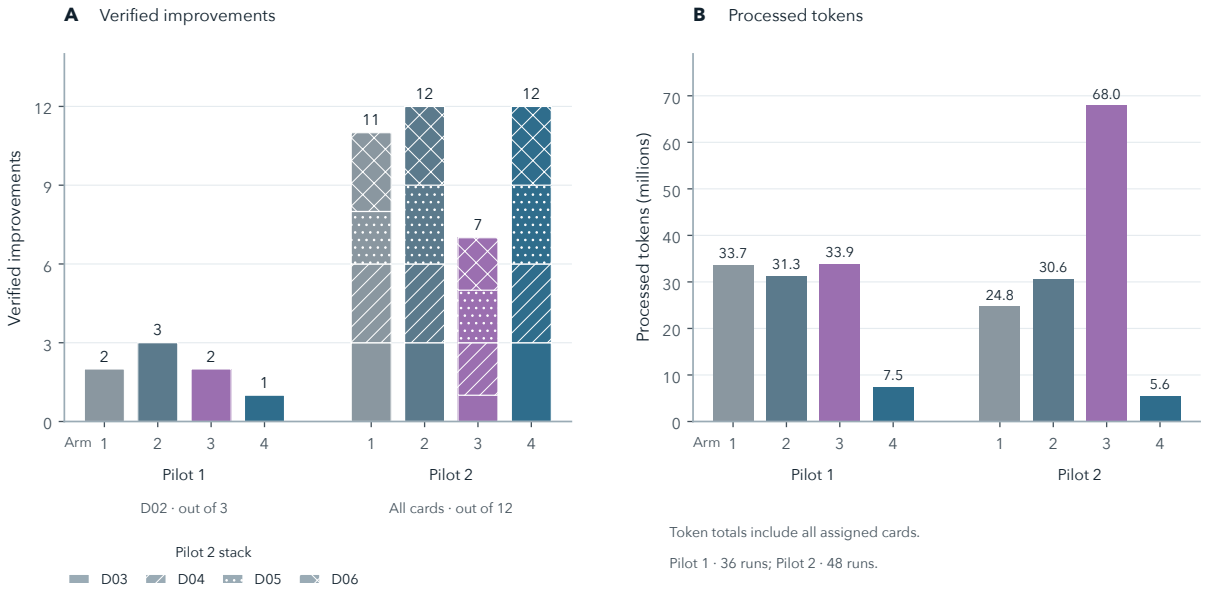


Figure 8. Pilots 1 and 2. **A:** verified improvements per arm (pilot 1: D02 only, out of 3; pilot 2: out of 12, stacked by card). **B:** processed tokens per arm. Pilot-2 numbers include the grader correction.

Neither run that reached a conclusion was wrong; both were verified. We still report the contrast exactly as registered.

The headline. Arm 4 minus arm 2 was **0.0**, with a bootstrap interval of $[0, 0]$. With four families this establishes neither equivalence nor superiority. Arm 4 beat arm 3 by +0.417 (descriptive). It matched the strongest coding arm while processing about **5× fewer tokens** (5.6M against 30.6M) and 12× fewer than arm 3, and it used the least wall time. No invalid or false improvement claim passed grading.

This table is corrected. As first written, it credited arm 2 with 11/12, giving Eidos a +0.083 lead. During pilot 3 we found that both graders had confirmed reproductions using a hard-coded checkpoint name, which wrongly failed arm 2's D03-r2 recipe (§7). Re-graded, that run verifies at +0.303. We withdrew the earlier claim publicly in an erratum.

The quality gap. The tie hides one thing. On D03 (shifted-image accuracy), the hardest card, arm 4's mean verified gain was **+0.225**, against +0.303, +0.290 and +0.294 for arms 1–3. On D04–D06, every successful run found near-ceiling solutions.

Arm 3. Arm 3 deserves a sentence of its own. Handing a coding agent the full Eidos toolkit over MCP made it *worse*: it had five token-budget stops, and 96.6% of its 68M tokens were cache reads, spent largely on uploads, schemas and plan repairs. The same capabilities that helped when built into the harness hurt when bolted onto a chat transcript.

6.4 Round 3: chasing discovery quality

The pilot-2 diagnosis read the D03 transcripts closely, and three things stood out:

- **A misread mechanism.** Arm 4 treated a shift in pixel *intensity* as a spatial *translation*, and so pursued augmentation and architecture changes. The best coding-agent solutions simply removed the offset.

- **Unused time.** Both diagnosed arm-4 runs stopped after one small batch, with 20–22 of their 30 minutes unused.
- **Unused tools.** The sweep and assessment tools were used in 0 of 8 arm-4 runs.

The diagnosis also found a bug: a scored baseline was not recognised in context and was run again.

Round 3 changed how Eidos explores (ADRs 0023–0025):

- **Incumbent and challengers.** The investigator holds an *incumbent* and runs challenger rounds until the exploration boundary. A challenger must beat it under a selection rule recorded before that round. Hold-backs protect time for reproduction and export.
- **Error diagnosis.** A reusable baseline error analysis reports data and error slices, and probes simple transformations, offset and gain among them, so that the mechanism can be diagnosed rather than guessed.
- **Typed sweeps.** Typed sweep templates put preprocessing and model families within reach without hand-written glue.
- **Compact MCP for arm 3.** A compact MCP workflow (upload, dependent batch, bounded status, submit) cut arm 3's overhead.
- **Provider resilience.** The investigator respects retry delays, stops cleanly on an exhausted quota, and records uncertain usage.

Four review rounds before the pilot (R14–R17) found real problems, and all were fixed before pilot 3:

- selection without a prior rule;
- held-out file paths reaching the diagnosis and sweep components;
- submission keys that did not bind the full request;
- a finalisation path that bypassed the hold-back.

6.5 Pilot 3: prospective cards, and one harness bug

Pilot 3 used the four prospectively built cards. The split is 60/20/20 with seed 20261111, and the baseline is a single linear layer trained with full-batch Adam for 80 epochs at learning rate 0.01, seed 11. Regression bars are 5% of the baseline's RMSE.

Card	Dataset	Metric	Seed-11 baseline	Useful gain
N01	Breast cancer	Accuracy	0.912	+0.02
N02	Diabetes	RMSE	178.01	−8.90
N03	Friedman 1, 2,000 samples	RMSE	11.04	−0.552
N04	Synthetic classification, 3,000 samples	Accuracy	0.688	+0.02

Pilot 3 completed all 48 assignments in 12 retained four-arm blocks. Another 19 block attempts were excluded, and all of them were re-run whole, with their records kept and their costs reported separately (§7):

- 16 were interrupted by provider quota outages, disk-space guards or the hosted-grading outage;
- 3 were excluded under addendum 1.

Arm	N01	N02	N03	N04	Total	Tokens	Wall
1. Coding agent	3/3	2/3	3/3	3/3	11/12	31.6M	11,688 s
2. Coding agent + toolkit	3/3	3/3	3/3	2/3	**11/12**	30.5M	9,635 s
3. Coding agent + Eidos (MCP)	3/3	1/3	3/3	1/3	8/12	59.7M	9,031 s
4. Eidos investigator	0/3	1/3	0/3	1/3	**2/12**	15.3M	14,659 s

The headline. Arm 4 minus arm 2 was **−0.75**, with a family-balanced bootstrap interval of [−1.00, −0.42], and arm 4 minus arm 3 was −0.50. This is the first interval in our pilots that excludes zero, and it does so in the wrong direction for Eidos. It is still exploratory: four fixed cards and three runs per cell.

Grading agreement. Local and hosted grading agreed on all 45 runs that both could grade, after the run-status gate. Against the raw hosted verdicts, where those were available, they agreed on 34 of 34. The three runs that stopped without an answer keep their status-gated failures and have no hosted grade.

Quality of the verified improvements. The pre-registered secondary outcome is the mean verified test gain per arm and card.

Card (gain)	Arm 1	Arm 2	Arm 3	Arm 4	Arm 4 − arm 2
N01 (accuracy)	+0.070	+0.076	+0.073	(none)	undefined
N02 (RMSE reduction)	125.13	125.17	125.28	125.24	+0.07
N03 (RMSE reduction)	9.95	9.86	9.82	(none)	undefined
N04 (accuracy)	+0.228	+0.258	+0.215	+0.192	−0.067

Where the investigator did verify, its gain matched the others on N02, where every successful run found the same ceiling, and was the smallest of the four on N04. These means condition on success and are undefined where an arm verified nothing, so they are not a measure of an arm's expected quality.

Resources. Arm 4 again processed the fewest tokens: 15.3M, about half of arm 2's 30.5M. It also used the most wall time, because its failing runs kept working until the deadline. Fewer tokens alongside failure is not efficiency. Arm 3 again processed the most (59.7M).

Across the retained runs, the arms processed 137.0M tokens in total, and the 19 excluded attempts another 74.6M. These are processed tokens, cache reads included, not billed cost, subscription quota or necessarily complete provider usage. Receipts are missing for three excluded attempts.

Over all 48 retained runs, the verdicts were 32 valid, 10 crashes, 3 token-budget stops, 1 timeout and 2 invalid outputs.

What broke. The diagnosis of the arm-4 failures (D3) found that the mechanism was not the model misbehaving. It was a rule in the harness: equation (6) in §3.4. Round 3 had asked the investigator to keep improving until the exploration boundary, which is exactly where the rule bites. In most of the failing runs the sequence was the same:

1. The model asked, validly, to read its latest results, with less than the 300 s request timeout left before the boundary. In the first run we diagnosed, 221–249 s remained; in others, as little as 10 s.
2. The runtime refused the follow-up request that would act on those results, because a full 300 s timeout no longer fitted.
3. It substituted an empty "wait for jobs" proposal.
4. The lab correctly refused that proposal, since nothing was running.

After six such refusals the investigation asked for an operator. The pilot has no operator, so the run ended.

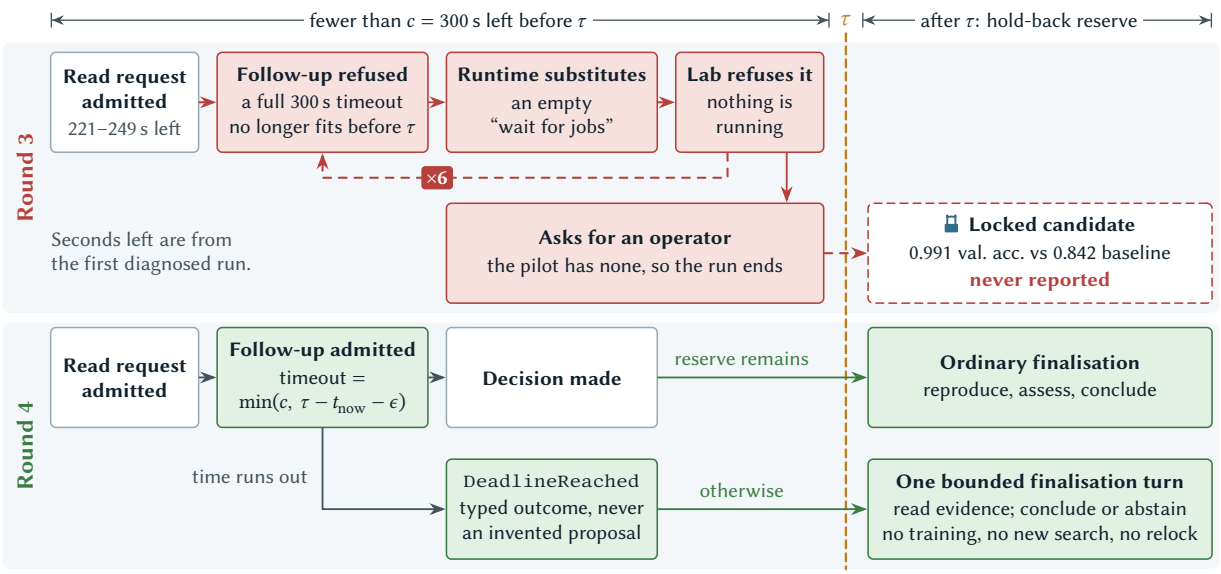


Figure 9. The round-3 defect and the round-4 repair. **Top:** with fewer than 300 s left, a valid read request never gets its follow-up decision; the substitute empty wait is refused six times, and the run ends without reporting its locked candidate. **Bottom:** round 4 bounds each request by the time actually left, and adds one bounded finalisation turn that can only read evidence and conclude.

Each of these runs held a locked candidate that was never reported; in one, it scored 0.991 on validation against a baseline of 0.842. This happened in **ten of the twelve** arm-4 runs: all three on N01 and N03, and two each on N02 and N04. Two of the ten had a twist:

- **N03-r2:** a second runtime limit, on decision tokens, produced the same kind of empty proposal.
- **N02-r3:** the model itself proposed one genuine wait before the runtime placeholders took over.

Three of the failing runs had also hit rate-limit errors earlier on, but each recovered, and none of those errors ended a run (addendum 2). The two arm-4 runs that reached a conclusion, N02-r1 and N04-r1, both verified.

The other arms' failures.

- **Arms 1 and 2:** one token-budget stop for arm 1 and one timeout for arm 2.
- **Arm 3:** two token-budget stops and two invalid outputs.

We caused arm 3's two invalid outputs. After addendum 4, both graders confirm a recipe using the checkpoint file name that the answer declares. Two arm-3 candidates declared `candidate_model.pt`, but their retraining code wrote `model.pt`, so confirmation found no file. Nothing in the answer schema or the prompts told agents that the two names must match. We handled it this way:

- we kept the recorded verdicts rather than re-grading after the fact;
- we disclose the ambiguity here;
- round 4 states the contract explicitly for every arm (ADR 0029).

The primary contrast is unaffected, and arm 3's count could rise by at most two.

Exposure checks. A literal scan of all 48 transcripts for network and package terms flagged 121 lines. We

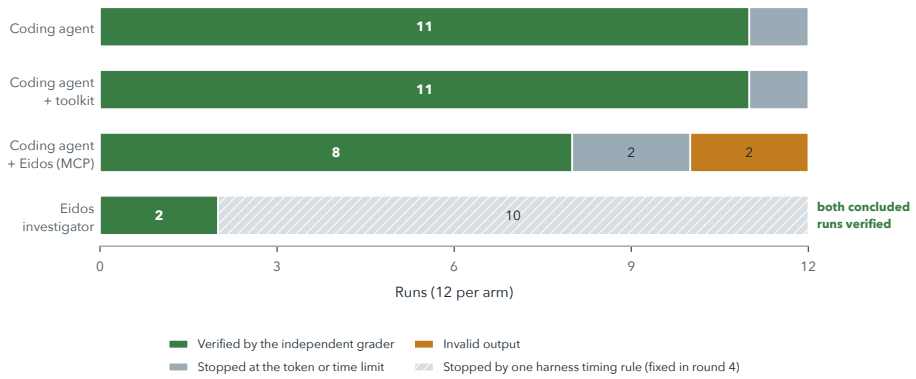


Figure 10. Pilot 3, how each run ended. Green runs were verified by the independent grader. Arm 4's ten hatched runs were all stopped by the same harness timing rule before they could report their locked candidate; it was removed in round 4. Arm 3's two invalid outputs came from a checkpoint-name ambiguity in our own instructions (below).

reviewed every one: they were local lab access, dependency-lock inspection and fragments of streamed text, and none led to an exclusion. A separate inspection of 153 stored arm-4 contexts found no exact match to the held-out test-split hashes. Neither check proves isolation, because full provider payloads were not available.

6.6 Round 4: no dead ends

Round 4 (ADRs 0026–0030) makes four main changes:

- **Deadline admission.** It replaces equation (6) with equation (7). A request is bounded by the time actually remaining, and a runtime that runs out of time reports that fact instead of fabricating a model proposal.
- **Bounded finalisation.** When an investigation would otherwise end without a conclusion, after repeated refusals or at the boundary with no usable decision, it gets **one** finalisation turn. That turn can read evidence and conclude, nothing more: no training, no new search, no rule or candidate changes. It runs inside the original budget, with its usage tracked separately, and it must report the locked candidate with its limitations or abstain honestly.
- **Rule preflight.** A selection rule is checked against the scorer's actual inputs before it is recorded. One pilot-3 run had bound its rule to the wrong input, so ordinary validation scores never counted as selection evidence.
- **Visible refusals.** Rejected model replies are kept in the transcript.

Algorithm 3 contrasts the old and new admission rules.

The first review of round 4 was not kind. It found:

- a **blocker**: the new default would have stopped *healthy* investigations from reproducing and confirming their candidates;
- three **majors**: held-out evidence could reach the model during finalisation, there was a loop at the boundary under the operator policy, and a crash path skipped the failure receipt.

All were sent back for fixing. Six more review rounds followed (R19–R24), and between them they found

Algorithm 3 Request admission near the boundary: the round-3 rule that stopped ten pilot-3 runs, and its round-4 repair.

```
function AdmitR3(t_now)                                          ▷ eq. (6)
    if τ − t_now > c then
        return Send(timeout = c)
    end if
    return EmptyWait()                  ▷ the trap: a placeholder the lab must refuse
end function

function AdmitR4(t_now)                                          ▷ eq. (7)
    if τ − t_now > ϵ then
        return Send(timeout = min(c, τ − t_now − ϵ))
    end if
    return DeadlineReached              ▷ a typed outcome, never an invented proposal
end function

function Finalise(I)
    if reserve remains then
        return OrdinaryFinalisation(I)                  ▷ reproduce, assess, conclude
    end if
    return BoundedTurn(I)   ▷ read evidence, conclude or abstain; no training, no search, no relock
end function
```

nine further major problems. Examples include gaps in how exploration contexts were isolated from held-out evidence, failures that still waited for an operator who would never come, and an accepted conclusion that could be written after the lab's deadline. Each was fixed before the next review, and a further review (R25) was in progress at the time of writing.

The final candidate has two more changes:

- it handles a second runtime limit found in pilot 3's last runs, on decision tokens (ADR 0030);
- it states the checkpoint-name contract for all arms (ADR 0029).

It passes 2,575 unit tests, and it replays all twelve recorded pilot-3 arm-4 failures through the corrected coordinator. **The replays show only that the old dead ends no longer occur.** They cannot show what a live model would have concluded instead. Where a corrected run diverges from history, the follow-up turns in a replay are synthetic and labelled as such. In one case, a missing reasoning-only reply is stood in for by variants that are equivalent to its receipts, not by reconstructed text.

Testing the repair on practice tasks led to two more changes:

- **Scope contract.** Autonomous finalisation can now validate an optional, operator-owned scope contract, keeping the durable guards and the original budgets.
- **Visible inference check.** All four arms now receive the same instruction to check, before locking, that the candidate's declared inference interface works in a fresh process on visible validation data, aligned by example ID. This is guidance. It is not enforced, it does not change the grader, and nothing yet shows that models follow it.

The candidate build (e503821) passes CI on the exact commit, including 719 integration tests. **Then an independent review found two blocking defects** in the candidate (e503821), both in how the coordinator routes the new typed outcomes:

- a decision cut off at the exploration boundary skips the ordinary finalisation phase, where reproduction happens, and goes straight to a one-shot conclusion turn;
- that turn allows a single model call, so one correctable reply ends the run as a controller abstention.

Together they could turn the Pilot-3 crash class into abstentions. Both were reproduced and fixed, together with further issues the same review raised: a strict freeze on the selected candidate once finalisation begins, one status vocabulary shared by all four arms, and a stricter test of when a provider has really recovered from a rate-limit error. The resulting build, a7c5d19, passed that independent review and its full CI on 12 November, including 754 integration tests.

How round 4 changed Eidos in practice.
We tested each candidate repair on seven practice tasks: the four Pilot-3 cards, plus D02–D04. Each time, the old and new builds ran side by side under the same controller, and we kept iterating until a build passed. Four candidates fell short; the fifth, e503821, did not.

Build	Runs completed	Improvements verified	Crashes
Pilot-3 build (257792a)	2 of 7	1	5
Round-4 build (e503821)	**7 of 7**	**6**	**0**

The failure that sank Pilot 3 is gone from these runs. Those were runs that dead-ended before reporting their best model.
What was left to fix. The traces also showed this. In 19 of 22 cases the build reached its conclusion through a one-shot turn triggered by a runtime limit: 15 deadline cut-offs and 4 malformed outputs. So it usually concluded without reproducing its candidate inside the run. It said so honestly, and left confirmation to the grader. The final build, a7c5d19, sends those cases back through ordinary finalisation, where reproduction happens.
These were the tasks we used while building the repair, with one run each. §6.7 reports the sealed Pilot-4 tasks, where the repaired harness met problems it had never seen.

What pilot 3 does and does not tell us.

1. **Observed.** The frozen round-3 build (257792a) verified 2 of 12 improvements, against 11 of 12 for the equipped coding agent, on four fixed cards. The contrast is −0.75 [−1.0, −0.42]. This is the measured result, and it stands.
2. **Fixed and tested since.** The failure paths behind it have engineering fixes, supported by source review, regression tests and replays of the recorded failures:
   - deadline and token bounds that report a typed outcome instead of an invented wait;
   - bounded finalisation;
   - selection-rule preflight;
   - held-out projection of model-facing context.
3. **Tested since.** Whether the repaired build verifies more improvements than the coding agent with the same toolkit, on tasks it has never seen. Pilot 4 asked this on sealed cards. The answer is inconclusive: 20 of 24 against 19 of 24, with an interval that spans zero (§6.7).

6.7 Pilot 4: sealed cards, two arms

Pilot 4 was the first test of the repaired harness on tasks it had never seen. It ran on 13 November 2026, from 05:00 to 13:24 UTC, and its result is **inconclusive**.

The build. The build that ran Pilot 4 is not the round-4 build of §6.6. That build, `a7c5d19`, failed its development gate, a paired check run between pilots (arm 4 only, one seed, seven development cards, with the frozen Pilot-3 build as the comparison). It concluded 6 of 7 runs. D02 crashed in the conclude-only wrap-up, because its final request had not returned when the decision deadline arrived. Round 5 (R5, ADR 0042) raised the conclusion reserve from 10% to 15% of the wall and split it into a read slot, an armed request timeout and one decision-only fallback request; the sizes follow the measured latency of 2,117 real requests. The gate for R5, G-R5, was registered before it ran, and it passed on build `86dfe57`: 7 of 7 runs concluded and were verified, with no dead ends and no crashes, against 1 of 7 for the frozen baseline. The fallback request never fired in that gate, and one seed on seven development cards that had already been used for tuning is development evidence, not an estimate of performance. The gap on N01 and N04, where the investigator reached 0.63 and 0.81 of the reference gain, remained. Pilot 4 ran build `86dfe57`.

Design. Pilot 4 compared arm 4 (the Eidos investigator) with arm 2 (a coding agent with the Eidos toolkit) on four new cards, N05–N08, built by the same prospective procedure as Pilot 3's. The cards were selected and sealed before any further runtime design work, and a separate reserve set, N09–N12, was sealed at the same time.

- **Qualification**: every card's reference recipe cleared its bar at all three qualification seeds, and none was tuned or replaced.
- **Commitment**: the cards are held privately, with salted hash commitments published in advance, so that anyone can check later that they did not change.

There were six runs per arm and card, in 24 randomised paired blocks (seed 20261113; within a block, arm 2 ran first; at most two blocks active at a time). That makes 48 assignments, and all of them stay in the denominator. Both arms used MiniMax-M3.1-Flash and the envelope of §5.1 (1,800 s, 8×10^6 processed tokens, one CPU thread). Both received the same seven-file task bundle and the same method guide. The verdict of record is the hosted two-VM grader plus the run-status gate (§5.4); the local grader was a pre-check. The estimator and the decision rule are the ones registered for the earlier pilots (§3.5). The primary contrast is arm 4 minus arm 2 in family-balanced success (equation (8)). A practically useful advantage is at least +0.10, and showing one needs the lower end of a two-sided 95% interval to exceed +0.10. The interval is a percentile bootstrap (2,000 draws, seed 71027) over cards and paired blocks. It is exploratory: the four cards are hand-selected, and each has one fixed dataset and split.

Results. The first table gives verified improvements by card and arm, the family-balanced success rate and the resources used. Tokens count cache reads, and arm 4's wall time excludes 394 s of infrastructure setup that is recorded separately.

Arm	N05	N06	N07	N08	Total	Family-balanced	Tokens	Wall
2. Coding agent + toolkit	4/6	5/6	4/6	6/6	**19/24**	0.792	78.9M	23,267 s
4. Eidos investigator	6/6	5/6	4/6	5/6	**20/24**	0.833	24.0M	32,152 s

Family-balanced success was 0.792 for arm 2 and 0.833 for arm 4. **The primary contrast, arm 4 minus arm 2, is +0.042, with an exploratory 95% percentile interval of [−0.208, +0.333].** Under the registered rule this is **inconclusive**. The point estimate is below the +0.10 margin and the interval spans zero, so Pilot 4

shows no advantage of practical size. It does not show equivalence either: the interval holds an advantage of 0.33 and a disadvantage of 0.21.

Quality of the verified improvements. The charter asks for the mean verified gain by arm and card, and for the per-card difference between arms. A mean is undefined, not zero, where an arm verified nothing on a card; here every cell has at least four verified runs. Gains are in each card's own metric (accuracy, or RMSE reduction, baseline minus candidate), so they are not comparable across cards.

Card (metric)	Verified, arm 2	arm 4	Mean gain, arm 2	arm 4	Difference
N05 (accuracy)	4/6	6/6	+0.1875	+0.1896	+0.0021
N06 (accuracy)	5/6	5/6	+0.4479	+0.4421	−0.0058
N07 (RMSE reduction)	4/6	4/6	297.64	297.48	−0.155
N08 (RMSE reduction)	6/6	5/6	3.788	3.744	−0.044

Arm 4 verified all six runs on N05 where arm 2 verified four. The arms tied on N06 (5/6) and N07 (4/6), and arm 2 verified all six on N08 where arm 4 verified five. Where both arms succeeded, the gains were essentially equal: on every card, arm 4's mean verified gain was within 1.4% of arm 2's. **Eidos found no larger improvements than the coding agent with the same toolkit.** These means condition on success, so they are not a measure of an arm's expected quality.

Resources. Arm 4 processed 24.0M tokens in total against 78.9M for arm 2, which is 3.3 times fewer. Per run, the median was 1.05M against 2.47M, and arm 4 used fewer tokens in 23 of the 24 paired blocks. Arm 4 used more wall time: 32,152 s in total against 23,267 s, and a median of 1,368 s per run against 838 s. Four arm-2 runs stopped at the 8M-token ceiling; no arm-4 run did.

Three cautions apply to the token comparison. First, arm 4's usage is flagged uncertain in 17 of its 24 run records, so its token total may be incomplete; arm 2's records carry no such flag. Second, the two arms reach the provider through different interfaces, and their token mixes differ: cache reads are 95% of arm 2's processed tokens (75.3M of 78.9M) and 3% of arm 4's (0.74M of 24.0M). Third, billed cost is unknown for all 48 runs, so a token ratio is not a cost ratio.

How the runs ended. Nineteen of arm 2's 24 runs verified. Four stopped at the token ceiling, and one (N06-r5) ended with an invalid output. Three of the four token-ceiling runs (N05-r1, N05-r2 and N07-r3) had left a candidate that the hosted grader verified, and the registered run-status gate counts them as failures. The other two (N07-r2, at the token ceiling, and N06-r5) left no answer file, so nothing went to the hosted grader. All 24 arm-4 runs ended with run status ok: none crashed, timed out or stopped at the token ceiling. In Pilot 3, ten of twelve arm-4 runs were stopped by one timing rule and recorded as crashes. Twenty of the 24 arm-4 runs verified. Three returned a valid answer that the hosted grade did not verify (N06-r2, N07-r2 and N08-r6), and one (N07-r5) was an invalid output at hosted grading. These are hosted verdicts only; we did not audit what each of those four runs claimed. There were no incidents and no replaced blocks in either arm, and the only model identifier served was MiniMax-M3.1-Flash.

How firm is the sign of the estimate? Not very. Two things about the estimate matter more than its size.

- **The run-status gate decides the sign.** Three arm-2 runs (N05-r1, N05-r2 and N07-r3) produced candidates that the hosted grader verified, but the runs exceeded the 8M-token budget, and the registered gate counts a run that exhausts its budget as a failure. Without the gate, arm 2 would have 22 of 24 (family-balanced 0.917) and the contrast would be −0.083. The gate is part of the verdict of record, registered before the first run, so we report +0.042. The point estimate changes sign without it, and the interval spans zero either way.

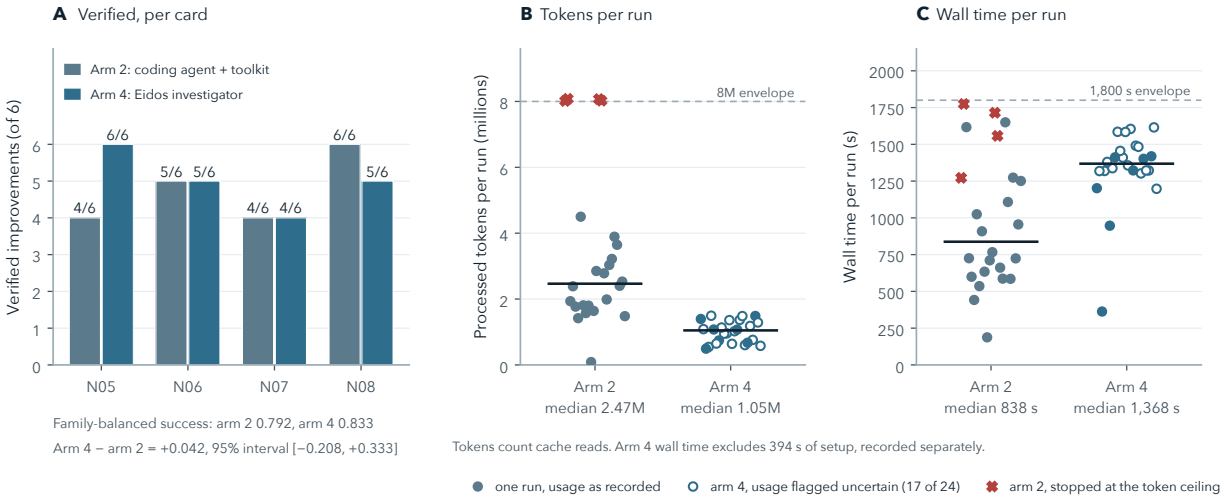


Figure 11. Pilot 4. **A:** verified improvements per card and arm, out of 6, with the family-balanced success rates and the registered contrast. **B:** processed tokens per run, cache reads included; crosses mark arm-2 runs stopped at the 8M-token ceiling, and open circles mark arm-4 runs whose usage is flagged uncertain. **C:** wall time per run; arm-4 times exclude setup. Horizontal bars are medians.

- **The local pre-check disagrees with the hosted grade on nine arm-4 runs.** The local grader marked each of them invalid; the hosted grader verified them. The local grading workspace had refused a write: each candidate's inference tried to write a receipt file next to its predictions, and the operating system returned a `PermissionError` (`[Errno 1] Operation not permitted`). The hosted grade is the verdict of record. A primary computed from the local pre-check alone would be −0.333 (arm 4: 11 of 24; arm 2: 19 of 24, the same locally and on the hosted grader). We do not present that as a result, because it measures a defect in the local pre-check, not the arms. Of the 46 runs that both graders judged, they agreed on whether the run verified for 37 and disagreed on nine, all of them arm 4. The other two runs left no answer to grade. Where both produced a numeric gain (32 runs), they agree on the verified verdict in all 32 and on the gain to within 5×10^{-6}. A tenth arm-4 run (N07-r5) met the same local error, and the hosted grader also recorded an invalid output, so there the two agree; the hosted record does not say whether the cause was the same.

What Pilot 4 does and does not tell us.

1. **Observed.** On four sealed cards, with six paired runs each, the repaired build (`86dfe57`) verified 20 of 24 improvements against 19 of 24 for the coding agent with the same toolkit. The contrast is +0.042, with an exploratory interval of [−0.208, +0.333]. Under the registered rule this is inconclusive. No arm-4 run crashed, timed out or stopped at the token ceiling, which is the failure class that dominated Pilot 3.
2. **Not shown.**
   - An advantage of practical size, a disadvantage, or equivalence.
   - Larger improvements: where both arms succeeded, the gains were equal to within 1.4%.
   - An effect of the new build. Pilots differ in cards, arms and build together, so Pilot 4 cannot be compared with Pilot 3 to say what the repair did.
3. **Fragile.** The sign of the point estimate depends on a registered status gate that counted three valid arm-2 candidates as failures. The four cards are hand-selected, share infrastructure and some mechanisms, and

each uses one fixed dataset and split. Twenty-four blocks are not twenty-four independent task families, and a common provider window can make runs dependent.
4. **What would settle it.** A matched comparison of old and new builds on a separately frozen design, using the reserve cards. Pilot 4 leaves that open.

The sealing is not full blinding. During pre-run tooling work, the engineers implementing and reviewing the tooling saw exact regression thresholds derived from the sealed cards' baseline metrics, and an unpublished local draft briefly carried them. The study models never received those values, and no outcome from the sealed cards informed any change made before the run, but the builders were not fully blind. There are further limits:

- the four cards share infrastructure and some mechanisms;
- each card reuses one fixed dataset and split across its six replicates;
- cards, arms and builds change together across pilots, so no comparison between pilots can attribute a difference to the harness alone.

Registration and the addenda. The charter, design, build and decision rule were registered before the first run: registration R (commit `f40f03c`) was pushed on 12 November 2026. Five addenda followed. Addenda 1 to 3b were each registered before they governed any later admission, and Addendum 4 was registered after all 48 assignments had resolved and affects only the final capture. The table in §7 lists the defects behind them, and Appendix B lists the commits.

- **Addendum 1: an earlier window.** On 13 November, after registration and before any model call, quota read or admission, the owner (the paper's author) moved the start of the admission window from 16 November 00:00 UTC to 13 November 05:00 UTC, to use the remaining weekly token-plan allowance. The design, cards, arms, envelope, decision rule and grader were unchanged.
- **Addendum 2: pay-as-you-go admission.** The owner added pay-as-you-go credit for the case where the weekly allowance ran out. A block could then be admitted on an owner-attested basis, recorded as `payg-attested`, when the interval allowance was positive and the weekly allowance was zero. Two consecutive quota or billing incidents paused admission. The attestation is a statement by the owner, not a measured provider balance.
- **Addenda 3 and 3b: an exhausted-week refusal and a tooling defect.** When the weekly allowance reached zero with status "exhausted", the admission policy refused the last block before it looked at the attestation. That was a design miss in our tooling, and Addendum 3 corrected it for that one case. The tooling that applied the correction then dropped an inherited binding from the freeze, and the guarded stage refused to start the last block (batch 13, decision 0013, 12:30 UTC, block N05-r6) before any worker or model call. Addendum 3b records such a refusal as an administrative non-admission (`refused-pre-worker`). The block stayed pending, no outcome of it existed, and it was admitted in batch 14 and run once, on the pay-as-you-go basis. The other 23 blocks were admitted on the token plan.
- **Addendum 4: capture validation, after all 48 assignments had resolved.** The final capture refused the directory of prompt-delivery receipts that the controller writes for every block, because its structural check did not know about it. Addendum 4 changed only that structural validation. The estimator, decision rule, status gate, verdict of record, denominator, incident rule, admission policy, model, budget, window and order were unchanged, and the addendum records that its changes were chosen from controller code and not from grades, verdicts or gains.

An erratum. Addendum 4 says the local pre-check recorded eight `PermissionErrors` on arm-4 runs. The retained local grades show ten: the nine disagreements above and N07-r5. The registered text is unchanged, and this note is the correction. No gate or number depends on it.

An interim look. While the last block was still pending, a descriptive table of the 46 settled assignments was compiled: equal-card success of 0.775 for arm 2 and 0.833 for arm 4, a difference of +0.058, with no interval. It was not the registered estimator, which is not run before all 48 assignments are reconciled. Addendum 4 says its changes were made before any outcome summary was computed; that statement is wrong, because this interim table already existed. A registered erratum (`ADDENDUM-4-ERRATUM.md`, commit `02e162d`) corrects it. Addendum 4's changes (recognising and verifying the prompt-delivery receipts, and a capture-only binding) were taken from the registered controller code, and an independent review confirmed that the estimator, status gates, verdict selection and denominators are byte-identical.

Independent audit. Before the READY record was written, a separate AI reviewer (a Claude Sonnet instance, not an outside party) audited the capture without opening any sealed material. It re-derived each of the 48 verdicts of record from the raw run statuses and hosted grades, and it recomputed the primary estimate (+0.0417, identical to the estimator record; it did not recompute the interval). It scanned all 48 transcripts for literal matches of `github`, `research/public`, `curl` and `pip`. There were 27 matching lines in 9 transcripts, all for `pip`, and it judged every one benign: 12 read-only `pip list` environment probes, 9 passages of model reasoning about the supplied `requirements.lock` or about the model's own probe output, 5 splits of the word "pipeline" across tokens, and one tool echo of public package versions. It found no leakage within that stated scope, which is literal term matches. A supplementary heuristic check of the file paths in tool calls found none outside a run's own directory. A literal scan does not prove isolation. The READY record was written on 13 November at 14:24 UTC and states the decision as inconclusive.

6.8 A side probe: a stronger model driving Eidos through MCP

Every arm in our study used the same modest model, so H2 was tested only with a weak driver. Would a frontier model, inside the same coding harness, use Eidos well? We ran a small, *unregistered* probe: arm 3 exactly as in pilot 2 (same prompt, tools, lab build, envelope and local grader), with one change, the model, which became Claude Opus 5.5 (`claude-opus-5-5`). One run each on the two pilot-2 cards where MiniMax's arm 3 struggled most.

Table 2. Arm 3 with a stronger driver. Opus 5.5: one run per card, pilot-2 local grader. MiniMax: the three pilot-2 runs per card, for context only; different day, lab instance and model.

Card	Driver	Verified	Held-out gain	Processed tokens	Wall
D03 (bar 0.01)	Opus 5.5	1/1	+0.278	0.65M	127 s
	MiniMax	1/3	+0.294	2.65M, 8.00M†, 8.10M†	383 s‡
D06 (bar 0.30)	Opus 5.5	1/1	+0.496	1.01M	154 s
	MiniMax	2/3	+0.496	5.48M, 3.26M, 8.16M†	765 s‡

† stopped at the 8M processed-token limit. ‡ wall time of the first verified run.

Both Opus runs verified, reproduced at fresh seeds (+0.278/+0.281 and +0.496/+0.496), and finished in about two minutes, using roughly 0.6–1.0M processed tokens and about $0.58–$0.64 each at list price. Neither showed MiniMax's failure modes: one invalid `purpose` value was fixed on the next call, and each run wrote a single answer that validated first time. The source-upload friction remained (about eight calls on D06), but Opus resolved it in a few steps rather than a loop.

What the probe does not show. Opus used Eidos sparingly: on D03 only 3 of 23 tool calls went through MCP, essentially to record the baseline, and it trained the winning candidate in its own shell. It also stopped at its first validated candidate with about 27 minutes left. So the probe suggests that the cost of Eidos-as-a-tool falls sharply with a stronger driver, but it does not show that Eidos-as-a-tool *adds* anything for such a driver. That needs the paired design we used for the weaker model, which is the natural next study (§9).

7 What went wrong, and what caught it

A harness for science needs a harness of its own. The table lists every defect that changed, or could have changed, a recorded outcome, and what caught it. We publish the addenda alongside the charters.

When	Defect	Who it could have favoured	Caught by	Resolution
Pilot 1	Arm 4's export wrote an invalid reproduce command	Against arm 4	Grading	Addendum; the whole block was re-run
Pilot 1	Null-card grader counted a permitted diagnosis as a false claim	Against the affected arms	Grading review	Grader corrected for all arms
Pilot 1	Incident rule read token counts as HTTP 5xx codes	Arm 4 (an automatic re-roll)	Controller audit	Re-roll stopped; attempt 1 kept
Before pilot 2	Assessor interval clustered only on seeds (73.7% coverage at nominal 95%)	Any candidate (false passes)	Independent review	Two-way interval (96.3%)
Before pilot 2	Stop/accept race, uncancellable decisions, double budget release	None systematic	Independent review	Fixed, with tests
Pilot 2	Recovered rate-limit notices ended coding-agent runs as incidents	Neither (symmetric)	Live monitoring	Addendum: terminal failures only
Pilot 2	Disk guard marked blocks as incidents before any arm started	None (no outcomes existed)	Live monitoring	Addendum; re-run
Pilots 2–3	**Both graders confirmed reproductions with a hard-coded** `model.pt`	**Arm 4 in pilot 2**: wrongly failed arm 2's only miss	A similar failure in pilot 3	Graders fixed; erratum; pilot 2 corrected to a tie
Pilot 3	Arm-3/4 charter code imported NumPy in an interpreter without it	None (no model call made)	Live monitoring	Addendum; three blocks re-run
Pilot 3	Arm-4 incident rule blamed a recovered early 429 for a later behavioural stop	**Arm 4** (would have re-rolled a real failure)	Record review	Addendum: provider failure must be terminal; audit of earlier pilots found no other case
Pilot 3	Arm 3 counted Claude Code's `<synthetic>` placeholder as a model change	None in practice	Record review	Addendum
Pilot 3	Hosted-grading runners could not resolve Ubuntu package hosts (GitHub later posted an incident); concurrent uploads lost a candidate	None (no verdicts produced)	Grading failures	Host pinning via DNS-over-HTTPS; serialised uploads; affected arms re-graded
Pilot 3	Internal-disk guards stopped blocks; the quota window ran out repeatedly	None (incidents re-run whole)	Live monitoring	Addenda 5–7: whole-block replacement, a bounded overnight resumption window and a fixed cut-off for new admissions

When	Defect	Who it could have favoured	Caught by	Resolution
Pilot 3	The declared-checkpoint rule from addendum 4 met an unstated contract: two candidates declared a name their retraining did not write	**Against arm 3** (two invalid outputs)	Failure diagnosis	Verdicts kept and disclosed; the contract is stated for all arms in round 4 (ADR 0029)
Pilot 4	The local pre-check could not write a receipt file in its grading workspace (`PermissionError`): ten arm-4 local grades, nine of them contradicting the hosted verdict	**Against arm 4**, in the local pre-check only	Comparison of local and hosted grades	Hosted grade kept as the verdict of record; the local-only contrast (−0.333) is a disclosed sensitivity, not a result; Addendum 4 counted eight, the records show ten (erratum, §6.7)
Pilot 4	The admission policy refused the last block when the weekly allowance read "exhausted", before it considered the owner's pay-as-you-go attestation	None (no block started, no outcome existed)	Live monitoring	Addendum 3
Pilot 4	The tooling that applied Addendum 3 dropped an inherited binding, so the guarded stage refused to start the last block	None (refused before any worker or model call)	Guard refusal	Addendum 3b: recorded as an administrative non-admission; the block was admitted in a later batch
Pilot 4	The final capture refused the controller's directory of prompt-delivery receipts	None (structure only; all 48 assignments had resolved)	Capture refusal	Addendum 4: structural validation only

Two of the earlier defects would have favoured Eidos. The grader defect *did*: it turned a tie into a win in a result we had already written up. Its fix then exposed an underspecified contract that cost arm 3 two runs in pilot 3. Fixing one grader rule changed which candidates a second, unstated rule would catch. Neither was found by looking for problems with Eidos. They were found because the charter forced every failure to be explained before the next block ran. We think this is the most transferable lesson in the paper. When an agent's work is graded by software, the grader is part of the experiment, and it needs the same scrutiny as the agent. The one Pilot 4 defect that touched a grade ran against arm 4, and only in the local pre-check.

8 Discussion

8.1 Why the architecture works

Three properties of the design show up directly in the data.

State lives in a ledger, not in a transcript. The investigator rebuilds a compact context from durable records at every turn, instead of carrying an ever-growing conversation. That is why it processed 4–12× fewer tokens than the coding agents in pilots 1 and 2, and 5.5× fewer than the best of them in pilot 2 while matching its score (Figure 1). In pilot 4 it again processed fewer tokens than arm 2 (24.0M against 78.9M, 3.3× fewer), with the caveats of §6.7: its usage is uncertain on 17 of 24 runs, and the two arms count cache reads very differently. The saving grows with the length of an investigation, because a transcript grows and a ledger summary does not.

Search is separated from confirmation. The investigator cannot grade its own work: a locked candidate must be reproduced at fresh seeds, and an assessor it cannot reach decides what counts. In pilots 2 and 3 every investigator conclusion that reached the grader was confirmed, 14 of 14, and in pilot 1 it made no false improvement claim. When the harness went wrong in pilots 1 to 3, it went wrong by *stopping*, never by letting an unverified claim through. For a system that will one day run unattended, that is the right way to fail. In pilot 4, 20 of its 24 runs verified and none crashed, timed out or stopped at the token ceiling; for the other four we report hosted verdicts only (§6.7).

Every decision is on the record. Each proposal, refusal, budget reservation and lock is a durable event. That made each pilot's failure diagnosable from its traces within hours: the pilot-1 loss led to round 2 and a tie at a fifth of the tokens; the pilot-3 timing rule was pinned down to the exact refusal sequence and the locked candidate it hid (in one run, 0.991 validation accuracy against a baseline of 0.842), and round 4 removed it. A coding agent's transcript records what it said; Eidos records what it decided and why it was allowed to.

The components help any agent. Eidos's components, handed to an ordinary coding agent as a toolkit with a written method (arm 2), made it the strongest or joint-strongest arm in every one of pilots 1 to 3: 3/3 on D02 in pilot 1 (2/3 without the toolkit), 12/12 in pilot 2 (11/12 without) and 11/12 in pilot 3 (a tie). In pilot 3 it also found the largest mean gains on two of the four cards, N01 and N04. In pilot 4 it faced only the investigator and verified 19 of 24 to the investigator's 20, with three further candidates that the grader verified counted as failures because their runs exceeded the token budget (§6.7). The toolkit does not force discipline; it makes the disciplined move the easy one, for example a scorer to call instead of one to write, and a paired report instead of an eyeballed comparison. That is the sixth principle of §4.1 at work. It also suggests a practical split: ship the components to the agents people already use, and keep the full harness for runs that must be trusted unattended. The differences are small and exploratory.

Efficiency, read carefully. If the per-token price of capable models stays high, the ledger's saving is not cosmetic. In pilot 3, though, arm 4's lower token count (about half of arm 2's) came mostly from runs that stopped early, and we do not count it as efficiency. In pilot 4 no arm-4 run crashed, timed out or stopped at the token ceiling, and it used about a third of arm 2's tokens, in 23 of 24 paired blocks, while verifying about as many improvements. That is closer to an efficiency result, but it is not a cost result: arm 4's usage is uncertain on 17 runs, the arms count cache reads very differently, it used more wall time (median 1,368 s against 838 s per run), and billed cost is unknown.

Who drives matters. The arm-3 result is the clearest finding in our data. The same toolkit that made the investigator efficient made a coding agent slow and expensive when exposed as MCP tools: 7/12 in pilot 2, with 68M tokens. A harness is not just a set of capabilities. It is a way of presenting them to a model, which is the agent-computer-interface lesson again (Yang et al., 2024b).

Is the harness redundant? Recent work argues that a strong model needs little scaffolding for ML engineering (Brilliantov et al., 2026), and that harness design mostly changes cost rather than accuracy (Fan et al., 2026). Pilot 2 is consistent with both. The best coding agent and the Eidos investigator verified exactly the same number of improvements, and the difference showed up in the bill. Pilot 4 is consistent with both again, and no more than that: 20 of 24 against 19 of 24 is inconclusive, and the investigator again used fewer tokens.

We read this as a statement about *which* quantity a training harness should be judged on. If the only target is "find a model that scores higher on the test set", a capable coding agent with good tools already does well on small tasks. The rigour layer then pays for itself in other currencies:

- **Fewer tokens per verified result.**
- **No unverifiable claims**, under graders that the agent cannot touch (Atinafu and Cohen, 2026).
- **Records that let a third party check the work** (Luo et al., 2025).

Those currencies become decisive as tasks get longer and more expensive, as models get smaller, and as agents gain the power to rewrite their own training loop (§8.3). Our pilots are too small to show that, and we flag it as the hypothesis the next study should test.

8.2 Why Eidos through MCP struggled

Hypothesis H2 failed for a reason we can measure. Our pilot-2 diagnosis read the transcripts of eight arm-3 runs and found that the losses were not scientific; they were plumbing.

- **The context grew with every tool call.** A coding agent carries its whole conversation forward, so every Eidos reply (charters, ledger entries, job receipts, schemas) is re-read on every later turn. 96.6% of arm 3's processed tokens were cache reads. Its first request already carried 13,405 input tokens, against 1,860 and 1,874 for arms 1 and 2; late in a run a single request reached 130,885–165,781 input tokens.
- **Integration loops replaced search.** One run (D03-r1) spent 91 model responses and 101 tool calls, many of them repairing command keys, purposes, source-upload schemas and artifact identifiers, then tried several evidence-plan forms before being, correctly, refused confirmation on selection data. Another (D06-r3) reached 100% validation accuracy, then spent the rest of its budget on upload and scope repair and six draft answers, ending at 8.16M tokens: a perfect candidate lost to bookkeeping.
- **The external surface was thinner than the internal one.** The study helper opened a fresh MCP session on every call, source authoring needed a separate HTTP upload with hand-written metadata, and the dependent batches the internal investigator uses (author → run → score) were not in the MCP vocabulary. Six of the eight inspected runs drove Eidos through shell commands rather than native MCP tools.

Formally, if a coding agent's context after t tool calls is $|C_t| \approx |C_0| + \sum_{i \le t} |o_i|$, where o_i is the i-th tool reply, then the tokens processed over a run grow as

$$K_{\text{transcript}} \;\approx\; \sum_{t=1}^{N} |C_t| \;=\; N|C_0| + \sum_{i=1}^{N} (N - i + 1)\,|o_i| \;=\; O(N^2 \bar{o}), \tag{10}$$

whereas the investigator rebuilds a context of roughly constant size $\bar{c}$ each turn, so $K_{\text{ledger}} \approx N\bar{c}$. Verbose tool replies $\bar{o}$ and many round trips N are exactly what arm 3 had; the quadratic term is why the same toolkit was cheap inside the harness and expensive outside it.

8.3 Where it got in the way

Every control is a failure surface. Discipline has a price, and our pilots paid it in two currencies.

- **Discovery quality.** In pilot 2 the investigator found the *right kind* of improvement less often on the hardest card; the coding agents explored more freely and found larger gains.
- **Refusals.** Validators refuse proposals, and a refused proposal costs a turn. In pilot 3 a timing rule turned refusals into a dead end.

Neither problem was a failure of the principles in §4. Both were failures of the machinery around them, and both show the same thing: in a harness, any control can become a new way to fail.

8.4 When the agent is inside the loop

The analyst is always looking. Reactive optimizers and weight-space programming make the evaluation problem harder, not easier. An agent that rewrites the optimizer while training runs is looking at the loss curve at every step (Bai et al., 2026). A model that writes weights directly can produce a network whose apparent competence comes from priors in the training data, not from the computation it was asked to implement (Ahmed, 2026).

What carries over. Each primitive of §4 still applies:

- **Charters.** What counts as an improvement is fixed before the agent starts.
- **Frozen evaluation.** The data the agent never touches is set aside in advance.
- **Separate assessors.** For weight-space programs, a model checker is a natural kind of assessor (Gross, 2026).
- **Reserved budgets.** Resources are reserved before work, so a run cannot spend its confirmation time on search.

How Eidos would host them. A reactive optimizer fits Eidos as an investigation whose candidate is an *optimizer program*: it would make a recipe claim, confirmed by retraining at fresh seeds. Weight-space programming produces checkpoint claims that need held-out confirmation like any other. We have not yet run either inside Eidos. We think that is the most interesting next step.

8.5 Open and smaller models

Model-agnostic by design. Eidos assumes very little about the model. It talks to providers through Anthropic-compatible and OpenAI-compatible adapters, including local servers. Our model gate was sobering, though: no local model of 14B parameters or fewer passed full conformance in the release checks, and the best (qwen2.5:14b) passed 4 of 8 scenarios.

Structure may matter more for smaller models. For smaller open models we expect the harness to matter *more*, not less. A model that cannot hold a long research transcript in its head benefits most from a harness that keeps the state for it, and a model whose self-assessment is weak benefits most from an assessor that does not depend on it. Anyone training new architectures, with any model, can put this kind of harness at the centre of the work, and get faster iteration and independently verified improvements. Testing that expectation directly is future work.

9 Limitations

The development cards are small, use public data and were built by the author. Pilot 3's prospective procedure limits, but does not remove, the risk of choosing cards that suit the investigator. N02 has only 89 test rows.

All arms used one model (MiniMax-M3.1-Flash, not snapshot-pinned), one machine and one provider. Arms 1–3 and arm 4 reach the provider through different interfaces.

The token envelope counts cache reads in full, which favours compact contexts. Billed cost is unknown.

The pilots are developer-run. The GitHub grader is a separate execution boundary, not an independent organisation.

The designs changed between pilots (pilot 4 also dropped arms 1 and 3), so comparisons across pilots are descriptive. Pilot 1's original records were partly reconstructed after a disk failure, and three of its blocks

have no transcript audit.

Every interval is exploratory. None of the pilots approaches the roughly 283 pairs needed for the pre-adopted decision rule; pilot 4, the largest, has 24.

Pilot 4 adds its own limits (§6.7). Its four cards were hand-selected, share infrastructure and some mechanisms, and each uses one fixed dataset and split, so its 24 blocks are not 24 independent task families, and a common provider window can make runs dependent. The builders had seen thresholds derived from the sealed cards' baselines, so the build was not developed under full blinding. The sign of the primary estimate depends on a registered run-status gate, and the investigator's token usage is flagged uncertain on 17 of 24 runs. The local pre-check was defective on arm 4, so we rely on the hosted grade alone, and the hosted grader is private. The capture tooling did not independently reconcile the identity of the arm-2 toolkit. Twenty-three of the 24 blocks were admitted on the token plan and one on an owner-attested pay-as-you-go basis. The independent audit was done by another AI reviewer, not by an outside party, and its transcript scan was limited to literal term matches.

10 Conclusion

We set out to test a simple idea. If an agent is a model plus its harness, then an agent that trains models needs a harness that knows how evidence works. We built one, ran it against coding agents with the same model, and redesigned it after every pilot. After one redesign it matched the best coding agent at a fifth of the tokens, and in pilots 2 and 3 every run in which it reached a conclusion was confirmed by a grader it could not touch. Its one setback, on pilot 3's prospective cards, came from a single timing rule in its own scaffold, which stopped ten runs before they could report. The durable record made that rule easy to find, and round 4 removed it: on the tasks used to test the repair, the old build crashed in 5 of 7 runs, while the repaired one completed all 7 and verified 6. Pilot 4, on four sealed cards and against the coding agent that has the same toolkit, was inconclusive: 20 of 24 verified improvements against 19 of 24, a difference of +0.042 with an exploratory 95% interval of [−0.208, +0.333]. The investigator found no larger improvements, used about a third of the tokens (with uncertain usage) and more wall time, and the sign of the estimate depends on a registered status gate. The repaired build did not repeat the failure of Pilot 3, but the pilot does not show that Eidos beats a coding agent that has its toolkit.

What the process caught. Along the way it caught bugs in our statistics, our graders and our incident rules, including one that had flattered our own system. Whatever one concludes about Eidos, we think that is the strongest argument for the approach.

The open question. The agents that rewrite optimizers and write weights are arriving quickly. The question of how we will know when they have actually improved a model deserves infrastructure, not just intuition.

Use of AI

AI coding assistants helped build Eidos and the study tooling, and assisted with analysis and drafting, under the author's direction. The author set the research question and every decision rule, and is responsible for every claim.

References

Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C. Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, 2021. URL https://proceedings.neurips.cc/paper/2021/hash/f514cec81cb148559cf475e7426eed5e-Abstract.html. arXiv:2108.13264.

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://arxiv.org/abs/2507.19457.

Nazmus Sakib Ahmed. Can an LLM actually reason and program about neural network's weights or is it just memorization on steroids? exploring "weight-space programming", 2026. URL https://arxiv.org/abs/2611.08070.

Samuel K. Ainsworth, Jonathan Hayase, and Siddhartha Srinivasa. Git re-basin: Merging models modulo permutation symmetries, 2022. URL https://arxiv.org/abs/2209.04836.

Sattam Altuuaim, Lama Ayash, Muhammad Mubashar, and Naeemullah Khan. PILOT: Policy-informed learned optimization for adaptive deep network training, 2026. URL https://arxiv.org/abs/2605.24570.

Marcin Andrychowicz, Misha Denil, Sergio Gomez, Matthew W. Hoffman, David Pfau, Tom Schaul, Brendan Shillingford, and Nando de Freitas. Learning to learn by gradient descent by gradient descent, 2016. URL https://arxiv.org/abs/1606.04474.

Anthropic. Introducing the model context protocol. Anthropic news, 25 November 2024, November 2024. URL https://www.anthropic.com/news/model-context-protocol.

Yonas Atinafu and Robin Cohen. RewardHackingAgents: Benchmarking evaluation integrity for LLM ML-engineering agents, 2026. URL https://arxiv.org/abs/2603.11337.

Xingjian Bai, Jia-Hao Liang, Jayden Teoh, and Vincent Sitzmann. Reactive optimizers, 2026. URL https://arxiv.org/abs/2610.33826. arXiv v1 24 Oct 2026, v2 4 Nov 2026; static version of the authors' blog post.

Joeran Beel, Min-Yen Kan, and Moritz Baumgart. Evaluating Sakana's AI scientist: Bold claims, mixed results, and a promising future? *SIGIR Forum*, 2025. URL https://arxiv.org/abs/2502.14297.

Bjarni Haukur Bjarnason, André Silva, and Martin Monperrus. On randomness in agentic evals, 2026. URL https://arxiv.org/abs/2602.07150.

Avrim Blum and Moritz Hardt. The ladder: A reliable leaderboard for machine learning competitions. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1006–1014, 2015. URL https://proceedings.mlr.press/v37/blum15.html. arXiv:1502.04585.

Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, et al. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems 3 (MLSys 2021)*, 2021. URL https://proceedings.mlsys.org/paper_files/paper/2021/hash/0184b0cd3cfb185989f858a1d9f5c1eb-Abstract.html. arXiv:2103.03098.

Kirill Brilliantov, Alejandro Hernández-Cano, and Emmanuel Abbé. How much of a harness does a strong agent need for autonomous ML engineering?, 2026. URL https://arxiv.org/abs/2609.40303.

Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11(70):2079–2107, 2010. URL https://www.jmlr.org/papers/v11/cawley10a.html.

Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, et al. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*, 2025. URL https://arxiv.org/abs/2410.07095.

Rujikorn Charakorn, Edoardo Cetin, Yujin Tang, and Robert Tjarko Lange. Text-to-LoRA: Instant transformer adaption. In *International Conference on Machine Learning (ICML)*, 2025. URL https://arxiv.org/abs/2506.06105.

Angelica Chen, David M. Dohan, and David R. So. EvoPrompting: Language models for code-level neural architecture search. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023a. URL https://arxiv.org/abs/2302.14838.

Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Yao Liu, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic discovery of optimization algorithms, 2023b. URL https://arxiv.org/abs/2302.06675.

Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024. URL https://arxiv.org/abs/2304.05128.

Dami Choi, Christopher J. Shallue, Zachary Nado, Jaehoon Lee, Chris J. Maddison, and George E. Dahl. On empirical comparisons of optimizers for deep learning, 2019. URL https://arxiv.org/abs/1910.05446.

Robert J. Connor. Sample size for testing differences in proportions for the paired-sample design. *Biometrics*, 43(1):207–211, 1987. doi: 10.2307/2531961.

George E. Dahl, Frank Schneider, Zachary Nado, Naman Agarwal, Chandramouli Shama Sastry, Philipp Hennig, Sourabh Medapati, Runa Eschenhagen, Priya Kasimbeg, Daniel Suo, Juhan Bae, Justin Gilmer, Abel L. Peirson, Bilal Khan, Rohan Anil, Mike Rabbat, Shankar Krishnan, Daniel Snider, Ehsan Amid, Kongtao Chen, Chris J. Maddison, Rakshith Vasudev, Michal Badura, Ankush Garg, and Peter Mattson. Benchmarking neural network training algorithms, 2023. URL https://arxiv.org/abs/2306.07179.

Thomas G. Dietterich. Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895–1923, 1998. doi: 10.1162/089976698300017197.

Jesse Dodge, Suchin Gururangan, Dallas Card, Roy Schwartz, and Noah A. Smith. Show your work: Improved reporting of experimental results. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2185–2194, 2019. doi: 10.18653/v1/D19-1224. arXiv:1909.03004.

Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248):636–638, 2015a. doi: 10.1126/science.aaa9375.

Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. Preserving statistical validity in adaptive data analysis. In *Proceedings of the Forty-Seventh Annual ACM Symposium on Theory of Computing (STOC)*, pages 117–126. ACM, 2015b. doi: 10.1145/2746539.2746580. arXiv:1411.2664.

B. Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. doi: 10.1214/aos/1176344552.

Erik S. and Barry Zhang. Building effective agents. Anthropic Engineering blog, 19 December 2024, December 2024. URL https://www.anthropic.com/engineering/building-effective-agents.

Run-Ze Fan, Zihao Zhang, Simin Ma, et al. An empirical study of harness design for coding agents, 2026. URL https://arxiv.org/abs/2609.20804.

Eduardo C. Garrido-Merchán. Zero-shot weight-space programming across a difficulty-graded ladder of two-player games, 2026. URL https://arxiv.org/abs/2611.09271.

Andrew Gelman and Eric Loken. The garden of forking paths: Why multiple comparisons can be a problem, even when there is no "fishing expedition" or "p-hacking" and the research hypothesis was posited ahead of time. Working paper, Department of Statistics, Columbia University, November 2013. URL https://sites.stat.columbia.edu/gelman/research/unpublished/forking.pdf.

Sean Goedecke. LLMs are not superhuman at reading and writing neural network weights. Blog post, 3 November 2026, 2026. URL https://seangoedecke.com/llms-are-not-superhuman-at-reading-and-writing-neural-network-weights/. Accessed 2026-11-11.

Dennis Gross. Weight-space programming of policies and their verification via model checking, 2026. URL https://arxiv.org/abs/2611.11604.

Siyuan Guo, Cheng Deng, Ying Wen, Hechang Chen, Yi Chang, and Jun Wang. DS-agent: Automated data science by empowering large language models with case-based reasoning. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, 2024. URL https://arxiv.org/abs/2402.17453.

David Ha, Andrew Dai, and Quoc V. Le. HyperNetworks, 2016. URL https://arxiv.org/abs/1609.09106.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*, 2018. URL https://arxiv.org/abs/1709.06560. arXiv:1709.06560.

Matthew Honnibal. The "Astra can read and write weights to improve Qwen" post is misleading. Blog post, 3 November 2026, 2026. URL https://honnibal.dev/blog/astra-read-write-misleading. Accessed 2026-11-11.

Jie Huang, Xinyun Chen, Swaroop Mishra, et al. Large language models cannot self-correct reasoning yet. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024. URL https://arxiv.org/abs/2310.01798.

Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. MLAgentBench: Evaluating language agents on machine learning experimentation, 2023. URL https://arxiv.org/abs/2310.03302.

Yue Huang, Zhangchen Xu, Yuchen Ma, et al. Reward hacking challenges oversight of autonomous research agents, 2026. URL https://arxiv.org/abs/2609.28614.

Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2212.04089.

Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, et al. AIDE: AI-driven exploration in the space of code, 2025. URL https://arxiv.org/abs/2502.13138.

Carlos E. Jimenez, John Yang, Alexander Wettig, et al. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024. URL https://arxiv.org/abs/2310.06770.

Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9): 100804, 2023. doi: 10.1016/j.patter.2023.100804. arXiv:2207.07048.

Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter. *Transactions on Machine Learning Research*, 2025. URL https://arxiv.org/abs/2407.01502.

Andrej Karpathy. autoresearch: AI agents running research on single-GPU nanochat training automatically. GitHub repository, 2026. URL https://github.com/karpathy/autoresearch. MIT license; README dated March 2026; accessed 2026-11-11.

Victoria Krakovna, Jonathan Uesato, Vladimir Mikulik, Matthew Rahtz, Tom Everitt, Ramana Kumar, Zac Kenton, Jan Leike, and Shane Legg. Specification gaming: the flip side of AI ingenuity. DeepMind blog, 21 April 2020, April 2020. URL https://deepmind.google/discover/blog/specification-gaming-the-flip-side-of-ai-ingenuity/.

Sydney Lewis. Same model, different harness: Different coding-agent results, 2026. URL https://arxiv.org/abs/2608.26218.

Zhongzheng Li, Tiancan Feng, Wenhao Li, Qingsong Ran, Shikun Feng, Xiaoyuan Zhang, Yue Wang, and Xiaoguang Zhao. OPTScientist: Multi-agent discovery of typed optimizer programs for transformer pretraining, 2026. URL https://arxiv.org/abs/2607.20486. arXiv submission history lists v1 as 2 Jun 2026 despite the 2607 identifier; re-check before citing.

Maximilian Lindholz, Richard Ruppel, Annette Milnik, Sebastian Halskov, Semil Eminovic, Charlie A. Hamm, Robin Schmidt, Anika Knupfer, Jana Hutter, Johanna P. Müller, Carolina Dominguez Aleixo, Jawed Nawabi, Jens Vogel-Claussen, and Tobias Penzkofer. Weight-space programming for medical imaging: Do we still need training data?, 2026. URL https://arxiv.org/abs/2611.12066.

Zachary C. Lipton and Jacob Steinhardt. Troubling trends in machine learning scholarship, 2018. URL https://arxiv.org/abs/1807.03341. Presented at ICML 2018: The Debates.

Fengshuo Liu, Ying Liu, Ruize Sun, Lie Luo, and Siyuan Guo. Coding agents have converged: Why the SWE-bench leaderboard can no longer order its top entries, and what to measure instead. In *International Conference on Advanced Data Mining and Applications (ADMA 2026), Special Session on Responsible Data Intelligence*, 2026. URL https://arxiv.org/abs/2609.17394.

Tennison Liu, Nicolás Astorga, Nabeel Seedat, and Mihaela van der Schaar. Large language models to enhance Bayesian optimization. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2402.03921.

Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery, 2024. URL https://arxiv.org/abs/2408.06292.

Ziming Luo, Atoosa Kasirzadeh, and Nihar B. Shah. The more you automate, the less you see: Hidden pitfalls of AI scientist systems, 2025. URL https://arxiv.org/abs/2509.08713. NeurIPS 2025 AI4Science workshop (Spotlight).

Alisia Lupidi, Bhavul Gauri, Thomas Simon Foster, et al. AIRS-bench: a suite of tasks for frontier AI research science agents, 2026. URL https://arxiv.org/abs/2602.06855.

Mitchell Marfinetz. Evolving deep learning optimizers, 2025. URL https://arxiv.org/abs/2512.11853.

Luke Metz, James Harrison, C. Daniel Freeman, Amil Merchant, Lucas Beyer, James Bradbury, Naman Agrawal, Ben Poole, Igor Mordatch, Adam Roberts, and Jascha Sohl-Dickstein. VeLO: Training versatile learned optimizers by scaling up, 2022. URL https://arxiv.org/abs/2211.09760.

Jaehyun Nam, Jinsung Yoon, Jiefeng Chen, Jinwoo Shin, Sercan Ö. Arık, and Tomas Pfister. MLE-STAR: Machine learning engineering agent via search and targeted refinement, 2025. URL https://arxiv.org/abs/2506.15692.

Deepak Nathani, Lovish Madaan, Nicholas Roberts, et al. MLGym: A new framework and benchmark for advancing AI research agents, 2025. URL https://arxiv.org/abs/2502.14499.

Aviv Navon, Aviv Shamsian, Idan Achituve, Ethan Fetaya, Gal Chechik, and Haggai Maron. Equivariant architectures for learning in deep weight spaces. In *International Conference on Machine Learning (ICML)*, 2023. URL https://arxiv.org/abs/2301.12780.

Robert G. Newcombe. Improved confidence intervals for the difference between binomial proportions based on paired data. *Statistics in Medicine*, 17(22):2635–2650, 1998. doi: 10.1002/(SICI)1097-0258(19981130)17:22<2635::AID-SIM954>3.0.CO;2-C.

Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018. doi: 10.1073/pnas.1708274114.

Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery, 2025. URL https://arxiv.org/abs/2506.13131.

Humza Nusrat and Omar Nusrat. Weight-space programming for auditing clinical decision networks, 2026. URL https://arxiv.org/abs/2611.05341.

Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Lariviere, Alina Beygelzimer, Florence d'Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program). *Journal of Machine Learning Research*, 22(164):1–20, 2021. URL https://www.jmlr.org/papers/v22/20-303.html.

Ben Rank, Hardik Bhatnagar, Ameya Prabhu, et al. PostTrainBench: Can LLM agents automate LLM post-training? In *Proceedings of the 43rd International Conference on Machine Learning (ICML 2026)*, 2026. URL https://arxiv.org/abs/2603.08640.

Anjali Rao and Nikhil Kamalkumar Advani. AI training manager: Bounded closed-loop control of adaptive training recipes, 2026. URL https://arxiv.org/abs/2606.29871.

Benjamin Recht, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar. Do ImageNet classifiers generalize to ImageNet? In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 5389–5400, 2019. URL https://proceedings.mlr.press/v97/recht19a.html. arXiv:1902.10811.

Philippe Rigollet and Jan-Christian Hütter. High-dimensional statistics: Lecture notes (mit 18.657), 2023. URL https://arxiv.org/abs/2310.19244.

Rebecca Roelofs, Vaishaal Shankar, Benjamin Recht, Sara Fridovich-Keil, Moritz Hardt, John Miller, and Ludwig Schmidt. A meta-analysis of overfitting in machine learning. In *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019. URL https://papers.nips.cc/paper/9117-a-meta-analysis-of-overfitting-in-machine-learning.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024. doi: 10.1038/s41586-023-06924-6. URL https://doi.org/10.1038/s41586-023-06924-6.

Samuel Schmidgall, Yusheng Su, Ze Wang, et al. Agent laboratory: Using LLM agents as research assistants, 2025. URL https://arxiv.org/abs/2501.04227.

Konstantin Schürholt, Dimche Kostadinov, and Damian Borth. Hyper-representations: Self-supervised representation learning on neural network weights for model characteristic prediction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. URL https://arxiv.org/abs/2110.15288.

Konstantin Schürholt, Boris Knyazev, Xavier Giró-i Nieto, and Damian Borth. Hyper-representations as generative models: Sampling unseen neural network weights. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022a. URL https://arxiv.org/abs/2209.14733.

Konstantin Schürholt, Diyar Taskiran, Boris Knyazev, Xavier Giró-i Nieto, and Damian Borth. Model zoos: A dataset of diverse populations of neural network models. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2022b. URL https://arxiv.org/abs/2209.14764.

D. Sculley, Jasper Snoek, Alex Wiltschko, and Ali Rahimi. Winner's curse? On pace, progress, and empirical rigor. In *ICLR 2018 Workshop Track*, 2018. URL https://openreview.net/forum?id=rJWF0Fywf.

Joseph P. Simmons, Leif D. Nelson, and Uri Simonsohn. False-positive psychology: Undisclosed flexibility in data collection and analysis allows presenting anything as significant. *Psychological Science*, 22(11):1359–1366, 2011. doi: 10.1177/0956797611417632.

Rikiya Takehi, Jayden Teoh, Alex Zhang, and Omar Khattab. Sparks of weight-space programming: LLMs can natively read & write neural network weights. Blog post, MIT CSAIL, 2 November 2026, 2026. URL https://rikiyat.github.io/blog/weight-space-programming/. Not on arXiv; experiment logs at github.com/RikiyaT/lm_understand-logs; accessed 2026-11-11.

Edan Toledo, Karen Hambardzumyan, Martin Josifoski, et al. AI research agents for machine learning: Search, exploration, and generalization in MLE-bench, 2025. URL https://arxiv.org/abs/2507.02554.

Dara Varam, Taraneh Aminosharieh Najafi, and David Atienza. Mixed signals: Weight-space programming for EEG and ECG, 2026. URL https://arxiv.org/abs/2611.13758.

Gaël Varoquaux. Cross-validation failure: Small sample sizes lead to large error bars. *NeuroImage*, 180:68–77, 2018. doi: 10.1016/j.neuroimage.2017.06.061. arXiv:1706.07581.

Sydney Von Arx, Lawrence Chan, and Beth Barnes. Recent frontier models are reward hacking. METR blog, 5 June 2025, June 2025. URL https://metr.org/blog/2025-06-05-recent-reward-hacking.

Xingyao Wang, Boxuan Li, Yufan Song, et al. OpenHands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*, 2025. URL https://arxiv.org/abs/2407.16741.

Zhangyang Wang, Peihao Wang, and Kai Wang. Position: Weight space should be a first-class generative AI modality, 2026. URL https://arxiv.org/abs/2605.18632.

Hjalmar Wijk, Tao Lin, Joel Becker, et al. RE-bench: Evaluating frontier AI R&D capabilities of language model agents against human experts, 2024. URL https://arxiv.org/abs/2411.15114.

Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning (ICML)*, 2022. URL https://arxiv.org/abs/2203.05482.

Yutaro Yamada, Robert Tjarko Lange, Cong Lu, et al. The AI scientist-v2: Workshop-level automated scientific discovery via agentic tree search, 2025. URL https://arxiv.org/abs/2504.08066.

Yutaro Yamada, Robert Tjarko Lange, Cong Lu, et al. Towards end-to-end automation of AI research. *Nature*, 651:914–919, 2026. doi: 10.1038/s41586-026-10265-5. URL https://arxiv.org/abs/2606.15497.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations (ICLR)*, 2024a. URL https://arxiv.org/abs/2309.03409.

John Yang, Carlos E. Jimenez, Alexander Wettig, et al. SWE-agent: Agent-computer interfaces enable automated software engineering, 2024b. URL https://arxiv.org/abs/2405.15793.

Xu Yang, Xiao Yang, Shikai Fang, et al. R&D-Agent: An LLM-agent framework towards autonomous data science, 2025. URL https://arxiv.org/abs/2505.14738.

Shunyu Yao, Jeffrey Zhao, Dian Yu, et al. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations (ICLR 2023)*, 2023. URL https://arxiv.org/abs/2210.03629.

Justin Young. Effective harnesses for long-running agents. Anthropic Engineering blog, 26 November 2025, November 2025. URL https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. TextGrad: Automatic "differentiation" via text, 2024. URL https://arxiv.org/abs/2406.07496.

Michael R. Zhang, Nishkrit Desai, Juhan Bae, Jonathan Lorraine, and Jimmy Ba. Using large language models for hyperparameter optimization, 2023. URL https://arxiv.org/abs/2312.04528.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, et al. Agentic context engineering: Evolving contexts for self-improving language models. In *The Fourteenth International Conference on Learning Representations (ICLR 2026)*, 2026a. URL https://arxiv.org/abs/2510.04618.

Wentao Zhang, Yang Young Lu, and Yuntian Deng. Interactive training: Feedback-driven neural network optimization, 2025. URL https://arxiv.org/abs/2510.02297. EMNLP 2025 System Demonstrations (per arXiv comment: EMNLP 2025 Demo).

Wentao Zhang, Xuanhe Pan, Han Zhou, Yang Lu, and Yuntian Deng. Interactive training 2: Auditable control plane for live model training, 2026b. URL https://arxiv.org/abs/2607.18314.

Bingchen Zhao, Despoina Magka, Minqi Jiang, et al. The automated LLM speedrunning benchmark: Reproducing NanoGPT improvements, 2025. URL https://arxiv.org/abs/2506.22419.

Allan Zhou, Kaien Yang, Kaylee Burns, Adriano Cardace, Yiding Jiang, Samuel Sokota, J. Zico Kolter, and Chelsea Finn. Permutation equivariant neural functionals. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2302.14040.

Adam Zweiger, Jyothish Pari, Han Guo, Ekin Akyürek, Yoon Kim, and Pulkit Agrawal. Self-adapting language models, 2025. URL https://arxiv.org/abs/2506.10943.

A Derivations

Optimism of the best of k (equation (2)). Let $\varepsilon_1, \dots, \varepsilon_k$ be independent $\mathcal{N}(0, \sigma^2)$. Then $\mathbb{E}[\max_t \varepsilon_t] \le \sigma\sqrt{2 \ln k}$, and the ratio tends to 1 as $k \to \infty$ (Rigollet and Hütter, 2023). Numerical integration gives 1.54σ, 2.25σ, 2.51σ and 3.24σ at $k = 10$, 50, 100 and 1000. The bound follows from Jensen's inequality applied to $\exp(\lambda \max_t \varepsilon_t)$, optimised over λ. For accuracy, ε is a centred binomial proportion and the Gaussian form is an approximation. Positively correlated candidates, the realistic case for variations of one recipe, have a smaller expected maximum. Figure 3 uses independent draws, so it shows the upper end.

Paired difference and its interval (equations 3–4). With per-example differences $d_i \in \{-1, 0, 1\}$, the variance of $\hat{\Delta}$ is estimated by $\widehat{\mathrm{Var}}(d)/n$. When candidates and baselines are trained with several seeds, examples are crossed with seeds. The two-way interval treats both as random effects, and resampling only seeds understates the variance (§6.2).

Sample size (equation (9)). Equation (9) is the planning approximation adopted in the study protocol, in the spirit of Connor's formula for paired binary outcomes (Connor, 1987; Newcombe, 1998). For paired binary outcomes with discordance probability ψ and true difference Δ, the variance of the per-pair difference is $\psi - \Delta^2$. A one-sided margin test against δ_0 at level $\alpha/2$ with power $1 - \beta$ needs $n \approx (z_{1-\alpha/2} + z_{1-\beta})^2(\psi - \Delta^2)/(\Delta - \delta_0)^2$. With $\psi = 0.40$, $\Delta = 0.20$, $\delta_0 = 0.10$, $\alpha = 0.05$ and $\beta = 0.20$, this is $7.84 \times 0.36/0.01 \approx 282$, which rounds up to 283. At 40 pairs, the half-width of a 95% interval is about $1.96\sqrt{0.36/40} \approx 0.19$.

Family-balanced estimator (equation (8)). Weighting families equally stops a card with more runs, or an easier card, from dominating. Bootstrap resampling is over families, and then over blocks within a family (2,000 resamples, seed 71027).

B Provenance

Item	Identifier
Pilot 1 charter, addenda 1–3	`71bcfc3`; `e4d25b4`, `36f9cba`, `e5e68fa`
Pilot 1 Eidos build	wheels from `ba79b91`
Round-1 fixes (I1–I5), integrated	`d34e717` (CI run 42946443787)
Pilot 2 charter, Eidos build, grader	`952e127`; `6258978`; grader `56a8889`
Pilot 2 addenda 1–3, erratum	`5583846`; `4840b55`, `543e8af`; `b715dd7`; erratum `83b0468`
Round 3 integrated	`42371dc`
Pilot 3 procedure, frozen cards, charter, build	`879c094`; `5bedf94`; `ecf85db`; wheels from `257792a`
Pilot 3 addenda 1–7	`e75f6ec`; `263dc80`; `d3341a7`; `83b0468`, `0cb98b8`; `9ada402`, `15f7ec6`; `35f6deb`, `06826e9`, `f504d79`; `ec49e72`
Pilot 3 final reconciliation	manifest SHA-256 `c8b85434`..., record rows `63c1ab2a`...
Grader fixes (checkpoint name, upload lock, DNS)	`d8106de`; `9d94e85`; `42de7ed`
Round 4	candidate `e503821` (CI runs 43184480156 and 43184488355, both green; PR #24); final round-4 build `a7c5d19`, which failed its development gate on D02; PR #24 was later merged to `main` by owner decision
R5	build `86dfe57` (ADR 0042; PR #27, merged as `f86fa10`); development gate G-R5, registered at `0ef3058` before it ran, passed

Item	Identifier
Pilot 4 registration chain	R `f40f03c` (charter v10.2, two-arm design); R2 `8873cc0` (addendum 1); R3 `902efdc` (addendum 2); R4 `1a260e6` (addendum 3); R5 `70fbcb6` (addendum 3b); R6 `6350616` (addendum 4)
Pilot 4 build, controller, graders	runtime and controller `86dfe57`; private grader `187205ac`; pinned scorer `f504d79`; investigator configuration SHA-256 `aacc14a0`...
Pilot 4 design and card commitment	`design.json` SHA-256 `5807bb88`... (seed 20261113); salted card commitment `3c76692f`..., published at `6700a11`
Pilot 4 final capture	manifest SHA-256 `6abc4301`...; estimator record SHA-256 `35dd082c`...; audit disposition SHA-256 `76ba0f53`...

Charters and addenda are in `docs/experiments/` of the public repository, https://github.com/Legend101Zz/Eidos. Pilot-3 documents and the pilot-2 erratum are on the branch `task/pilot-3-tooling` until it is merged (not merged at the time of writing; draft PR #22). The hosted grader repository is private at the time of writing, because it also holds the sealed task sets for later pilots. Its Actions logs are retained, and it will be published once those studies are complete. The Pilot-4 charter, addenda 1–4, design and registration record are in `docs/experiments/pilot-4/` at commit `6350616`, on the branch `task/pilot-4-registration`. Design decisions are ADRs in `docs/decisions/` (cited here: 0020–0030 and 0042).

C Reproducibility checklist

- **Code:** the Eidos source and study tooling are public; the frozen builds for each pilot are listed in Appendix B.
- **Data:** every card uses public data (scikit-learn datasets or public generators). Splits and seeds are fixed in the card bundles.
- **Model:** MiniMax-M3.1-Flash via the provider API, with the served model identifier logged per run. Not snapshot-pinned.
- **Envelope:** 1,800 s, 8×10^6 processed tokens (cache reads counted), one CPU thread.
- **Grading:** the local scorer, plus a two-VM GitHub grader (`Legend101Zz/eidos-grading`, private until the pilots that use its sealed task sets are complete). The workflow logs are retained.
- **Analysis:** `study/analyze.py`, with bootstrap seed 71027. The Pilot 4 estimator is the registered `analyze_results.py` in `research/public/development/`, run only after an exact 48-assignment reconciliation; the figure and tables of §6.7 are generated from the reconciled capture by `src/make_p4_figure.py`.
- **Not provided:** the billed cost of each run, which the provider's token plan does not report.